A PROJECT REPORT

On

Movie Recommendation System: NLP Based Recommender System

Submitted in partial fulfilment of the requirements to

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA

For the award of the degree

BACHELOR of TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

Submitted by

BATTULA VISHNU (22JK1A0527)

Under the Guidance of

K SRINIVAS REDDY

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
KITS AKSHAR INSTITUTE OF TECHNOLOGY :: YANAMADALA
(Approved by AICTE, New Delhi Affiliated to JNTUK Kakinada)
2022 – 2026

KITS AKSHAR INSTITUTE OF TECHNOLOGY

Approved by AICTE, Accredited by NAAC &Affiliated to JNTUK - Kakinada.

opp. Katuri Medical College, Yanamadala, Andhra Pradesh 522019

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

KITS AKSHAR INSTITUTE OF TECHNOLOGY

2022 – 2026

CERTIFICATE

This is certify that this Project work titled **"Movie Recommendation System: NLP-Based Recommender System"** is the bonafide work of BATTULA VISHNU(22JK1A0527),who carried out the work under the supervision and submitted in the partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science & Engineering** from **JNTU Kakinada** during the academic year 2022-2026

Project Guide

K SRINIVAS REDDY

Assistant Professor

Head of the Department

Dr. G. GURU KESAVA DAS M. Tech, Ph. D

Professor&HOD

External Guide

DECLARATION

We are here by inform that this main project entitled **"Movie Recommendation System: NLP-Based Recommender System"** has been carried out and submitted in partial fulfilment for the award to the degree of Bachelors of Technology in Computer Science & Engineering to **Jawaharlal Nehru Technical University Kakinada** under the guidance of **Mr. K SRINIVAS REDDY**, Assistant Professor Dept of Computer Science & Engineering. The work embodied in this project work is original and has been submitted in part or full for any degree of any other university.

BATTULA VISHNU 22JK1A0527

ACKNOWLEDGEMENT

We express our profound gratitude to Dr. GURU KESAVA DAS GOPISETTY, M.E (CSE), Ph.D. (CSE), Professor & Head of the Department of Computer Science and Engineering (CSE), for his invaluable guidance and unwavering support as our internal guide. His meticulous supervision and insightful advice have been instrumental in the successful completion of our B. Tech main project.

We are deeply indebted to Dr. G. GURU KESAVA DAS for providing us with all the necessary resources and encouragement throughout our project journey. We extend our heartfelt thanks to the college management for their continuous support and the excellent facilities provided, enabling us to carry out our project efficiently. Our sincere gratitude also goes to Dr. K. RAMA KOTAIAH, Principal, for granting us permission to undertake our main project.

Furthermore, we express our appreciation to our Director, Dr. G. SAMBASIVA RAO, our Secretary, Sri K. SEKHAR, and our Chairman, Sri K. SUBBA RAO, for their guidance and assistance during the course of our work. We are immensely thankful to the faculty and staff of the Department of Computer Science and Engineering for their valuable time, encouragement, and technical support.

Finally, we extend our deepest gratitude to our parents, friends, and well-wishers, whose unwavering support and motivation have been a driving force in our academic journey.

Institute Vision

To be a recognized premier institution in engineering education, research and application of knowledge to benefit society.

Institute Mission

- Impact technical knowledge at all levels to transform the engineers capable to address the challenges.
- Inculcate competencies for all-round development to meet industrial, technical and leadership skills.
- Provide learner-centric ambiance with collaborations.
- Promote creativity, knowledge development, ethical values and inter-personal skills.

Department of Computer Science & Engineering

Vision

To be a centre of excellence in computer engineering education, empowering Graduates as highly skilled professionals.

Mission

- **DM1:** Provide a learning Platform with emphasis on technical excellence.
- **DM2:** Organize trainings to meet industrial and emerging technologies.
- **DM3:** Install the drive for higher learning and research.
- **DM4:** Inculcate leadership and entrepreneur skills with industry collaborations.

Program Specific Outcomes (PSOs)

PSO1: Identify the data; Design suitable algorithm by using Latest software or real Time applications.

PSO2: Understanding the evolutionary changes in computing process knowledge of context aware applicability of paradigms and meet the challenges of the future.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Analyze, design, develop and test the software products and provide creative, sustainable solution.

PEO2: Use modern tools, techniques to analyze and provide solutions to the industrial problems.

PEO3: Demonstrate the managerial and leadership skills to become successful professionals.

Project Outcomes

PO-1 EngineeringKnowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO-2 Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO-3 Design/development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations

PO-4 Conduct Investigations of Complex Problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO-5 Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO-6 The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO-7 Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts.

PO-8 Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO-9 Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO-10 Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO-11 Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO-12 Life-Long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Project Course Outcomes

C424.1: Review the literature and develop solutions for problem statement

C424.2: Implement hardware and/or software requirements for identified problems

C424.3: Design the code for the problem statement **C424.4:**

Test and analyze the modules of planned project.

C424.5: Write technical report and deliver presentation.

C424.6: Applyengineering and management principles to achieve project goal.

Course Outcomes – Program Outcome correlation

	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2
C424.1	3	2		2		2	3						3	
C424.2	3	2		2		2			3	2	1		2	2
C424.3		3			3							2	1	
C424.4	2	2	3	2	3		2	1	3	2			3	
C424.5			3		2		2		3		2	2	3	2

C424.6						3				2		2		3

3: High 2: Medium 1: Low

CO-PO Mapping with Reasons:

C424.1 is mapped with PO1, PO2 and PO4, PO6, PO7 as basic knowledge of Engineering and problem Analysis activities are highly essential to conduct examinations on existing systems which have been using in industries as a part of and to define the problem of proposed system.

C424.2 is mapped with PO1, PO2, PO4 and PO6, PO9, PO10, PO11 as for identification, gathering analysis and classification of requirements for the proposed system, basic knowledge of engineering and Analysis steps along with complex problem analysis through the efforts of team work in order to meet the specific needs of the customer.

C424.3 is mapped with PO2, PO5 and PO12 as to conduct the literature review and to examine the relevant systems to understand and identify the merits and demerits of each too enhance and develop the proposed as per the need.

C424.4 is mapped with PO1, PO2, PO3, PO4, PO5 and PO7, PO8, PO9, PO10 because modularization and design of the project is needed after requirements elicitation. For modularization and design of the project, Basic knowledge of Engineering, Analysis capabilities, Design skills and communication is needed between team members as different modules are designed individually before integration.

C424.5 is mapped with PO3, PO5, PO7, PO9, PO11 and PO12 as to construct the project latest technologies are needed. The development of project is done individually and in groups with well-defined communication by using the engineering and management principles.

C424.6 is mapped with PO6, PO10 and PO12 because during and after completion of the project, documentation is needed along with proper methods of presentation through understanding and application of engineering and management principles, which in turn needs well defined communication between the team members with all the ethical values.

CO-PSOs Mapping with Reasons:

C424.1 is mapped with PSO1 as examining of existing systems and identification of the problem is a part of Application Development activity and identification of evolutionary changes in latest technologies.

C424.2 is mapped with PSO1 and PSO2 as identifying and classifying the requirements is a part of Application development and evolutionary computing changes and also follows ethical principles.

C424.3 is mapped with PSO1 as review of literature is a part of application development activity by recognizing the computing technologies and their evolutionary changes.

C424.4 is mapped with PSO1 because modularization and logical design is also a part of Application development and follows computing changes using Deep learning technology.

C424.5 is mapped with PSO1, PSO2 as Testing, Development and Integration of project activities are part of Application development and follows ethical principles.

C424.6 is mapped with PSO2 as for project documentation and presentation; the project team members apply the professional and leadership qualities.

INDEX

ABSTRACT i

CHAPTER / TOPIC **PAGE NO.**

CHAPTER 1 INTRODUCTION 01-11

1.1 Background and Motivation 02-04

1.2 Problem Statement 04-06

1.3 Objectives of the Study 06-07

1.4 Scope of the Project 08-09

1.5 Organisation of the Report 09-11

CHAPTER 2 LITERATURE SURVEY 12-18

2.1 Overview of Recommendation System 13-14

2.2 Content-Based Filtering 14

2.3 Collaborative Filtering 15-16

2.4 Hybrid Approaches 16

2.5 NLP Techniques in Recommendation 17

2.6 Summary Table 18

CHAPTER 3 SYSTEM ANALYSIS 19-23

3.1 Survey of Existing Tools 20

3.2 Identified Limitations 21

3.3 Proposed System Overview 22

3.4 Comparative Feature Analysis 23

CHAPTER 4 SYSTEM REQUIREMENTS 24-28

4.1 Functional Requirements 25

4.2 Non-Functional Requirements 26

4.3 Hardware Requirements 27

4.4 Software Requirements 28

CHAPTER 5 SYSTEM STUDY 29-34

5.1 Feasibility Study 30-32

5.2 Feasibility Analysis 32-34

CHAPTER 6 SYSTEM DESIGN 35-43

6.1 System Architecture 36

..... 34

6.2 UML Diagrams..... 3836

6.3 Class Diagram 38

6.4 Sequence Diagram..... 39

6.5 Activity Diagram 40

6.6 Collaboration Diagram 43

CHAPTER 7 INPUT AND OUTPUT DESIGN 44-47
7.1 Input Design 45-46
7.2 Output Design 47
CHAPTER 8 IMPLEMENTATION 48-58
8.1 Development Environment 49-50
8.2 Module Description 50-55
8.3 NLP Pipeline 56-58
CHAPTER 9 SOFTWARE ENVIRONMENTS 59-77
9.1 Python 60-73
9.2 Source Code 74-80
CHAPTER 10 RESULTS / DISCUSSIONS 81-101
10.1 System Test 82-95
10.2 Screenshots 96-101
CHAPTER 11 CONCLUSION 102
11.1 Summary of Findings 103
11.2 Future Scope 104
CHAPTER 12 REFERENCES 105-107

LIST OF FIGURES

Figure No.	Figure name	page
Fig 1	System architecture	36-37
Fig 2	UML Diagrams	38-39
Fig 3	Class Diagram	40
Fig 4	Sequence Diagram	41
Fig 5	Activity Diagram	42
Fig 6	Colloboration Diagram	43
Fig 9.1-9.8	Python Download	60-73
Fig 10.1-10.6	Screenshorts	93-98

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CSV	Comma-Separated Values
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
EDA	Exploratory Data Analysis
GMM	Gaussian Mixture Model
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
ML	Machine Learning
NaN	Not a Number
PCA	Principal Component Analysis
REST	Representational State Transfer

RT	Rotten Tomatoes

Movie Recommendation System: NLP Based Recommender System

ABSTRACT

The challenge of discovering movies that genuinely match a viewer's taste has grown harder as streaming platforms now carry catalogs of tens of thousands of titles. Browsing by genre is too coarse, popularity charts recycle the same well-known titles, and services that use collaborative filtering are useless to anyone without an established viewing history. This project Movie Recommendation System: NLP Based Recommender System tackles this problem by building a content-based recommendation engine that understands films through the language used to describe them.

The system processes the TMDB 5000 Movie Dataset, extracting genre tags, thematic keywords, plot overviews, top-billed cast members, and director credits for each of the approximately 4,800 films. These features are merged into a single descriptive tag string per film and then cleaned using Natural Language Processing — tokenisation, lowercase normalisation, and English stopword removal via NLTK. The cleaned corpus is vectorised

using TF-IDF (Term Frequency-Inverse Document Frequency) with up to 10,000 features and 1-2 word n-grams. Cosine similarity is computed across all film pairs, producing a 4,800 x 4,800 similarity matrix serialised to disk and loaded instantly on subsequent runs.

The recommendation engine is exposed through a Flask web application with a dark- themed, responsive interface. Users type any movie title into the search bar — aided by live autocomplete — and receive the ten most similar films in under a second. Each recommendation card displays the movie poster (fetched from the OMDb API), title, and similarity score. A detail modal provides additional context: full plot, cast, director, IMDB rating, Rotten Tomatoes score, runtime, content rating, awards, and a direct YouTube trailer search link.

The project demonstrates that NLP-driven content-based filtering can deliver contextually meaningful recommendations without any user data, making it immediately useful from the very first query. The architecture is modular, the pipeline is fully reproducible, and the system is ready for deployment with minor infrastructure hardening.

Keywords: Content-based filtering, NLP, TF-IDF, cosine similarity, Flask, movie recommendation, TMDB dataset, OMDb API, Python, scikit-learn, NLTK.

CHAPTER-1
INTRODUCTION

1. INTRODUCTION

1.1 Background and Motivation

In the modern digital entertainment era, online streaming platforms have transformed the way people consume movies and television content. Services such as Netflix, Amazon Prime Video, Disney+, and YouTube offer thousands of titles to viewers across the world. With such a vast collection of entertainment choices, users often face a common problem known as **choice overload**. Instead of enjoying the convenience of unlimited content, many viewers spend a considerable amount of time searching for something interesting to watch. This challenge has made recommendation systems one of the most valuable technologies in the streaming industry today.

Every major streaming platform now includes personalized sections such as *"Because you watched…"*, *"Recommended for You"*, or *"Top Picks."* These suggestions are not random; they are generated by intelligent recommendation engines that analyze user interests and match them with suitable content. Behind these simple recommendations lies a sophisticated

computational system capable of processing large amounts of data and delivering results within seconds. The objective of these systems is to reduce the time users spend browsing, improve customer satisfaction, and increase user engagement on the platform.

The scale of this problem is significant. Modern platforms host thousands of movies, TV shows, documentaries, and short videos. For example, Netflix alone provides access to more than 17,000 titles worldwide. For an average viewer, manually searching through such a large catalogue can be confusing and time-consuming. Research and user behavior studies indicate that many users spend several minutes deciding what to watch, and in some cases abandon the platform without selecting anything. This directly impacts viewer satisfaction and platform retention rates. Therefore, an effective recommendation system becomes essential for both users and service providers.

Recommendation systems generally operate using two major paradigms: **Collaborative Filtering** and **Content-Based Filtering**.

Collaborative Filtering recommends items based on the collective behavior of multiple users. It works on the assumption that users who shared similar preferences in the past are likely to share similar interests in the future. For example, if two users both enjoyed movies like The Godfather and Goodfellas, then the system may recommend other crime or drama films liked by one of them to the other. This method is highly effective in largescale commercial systems because it uses real user interactions such as ratings, clicks, watch history, and viewing duration.

However, collaborative filtering also has certain limitations. It requires a large amount of user data to perform well. New users with no history face the **cold-start problem**, where the system cannot make accurate recommendations due to lack of information. Similarly, newly added movies or lesser-known titles may not be recommended often because there is insufficient interaction data. In addition, some users may have privacy concerns regarding the collection and use of personal viewing history.

The second major approach is **Content-Based Filtering**, which focuses on the characteristics of the items themselves rather than relying on user behavior. In the context of movies, this includes attributes such as genre, cast, director, keywords, themes, language, release year, and storyline descriptions. If a user likes an action-thriller movie starring a certain actor, the system may recommend other movies with similar content attributes. This method is especially useful when user history is limited or unavailable.

Content-based systems offer several advantages. They can recommend new or less popular movies immediately because recommendations are based on metadata rather than popularity. They also preserve privacy more effectively because they do not depend heavily on tracking

user behavior. Furthermore, recommendations are easier to explain. For instance, a system can state that a movie was suggested because it shares the same genre, actors, or storyline elements as a previously selected title. This transparency increases user trust and satisfaction.

The project **CineMatch** is motivated by the strengths of the content-based recommendation paradigm. It specifically explores the use of **Natural Language Processing (NLP)** techniques to analyze textual movie metadata. Modern movie datasets contain rich textual information such as overviews, plot summaries, taglines, genres, and keywords. NLP enables computers to process this text, identify important terms, and measure similarity between movies. By converting textual descriptions into numerical vectors using methods such as TFIDF (Term Frequency–Inverse Document Frequency), the system can compare movies mathematically and recommend titles with similar content.

The motivation for selecting this approach is practical and relevant to real-world scenarios. A content-based NLP recommendation system does not require existing user ratings or historical interaction logs, making it easier to deploy for new applications, startup platforms, or academic projects. It is immediately useful because users can receive recommendations simply by selecting a movie they already like. There is no need for lengthy registration processes, extensive profiling, or data collection.

Another strong motivation is privacy awareness. In today's digital world, users are increasingly concerned about how their personal data is collected and utilized. Since CineMatch relies primarily on movie metadata rather than personal watch history, it reduces privacy risks while still delivering personalized suggestions. This makes the approach suitable for privacy-conscious environments.

Finally, the project aims to demonstrate how machine learning and NLP techniques can solve everyday problems in an intuitive way. By building CineMatch, the project combines data science, software development, user interface design, and recommendation algorithms into a practical application. It provides users with faster decision-making, improves movie discovery, and enhances the entertainment experience. Therefore, the development of CineMatch is both academically valuable and practically beneficial in the growing field of intelligent recommendation systems.

1.2 Problem Statement

The rapid growth of digital entertainment platforms has created an environment where users have access to an enormous number of movies, web series, documentaries, and other forms of video content. While this abundance of choice appears beneficial, it has also introduced a

significant challenge: users often struggle to discover films that truly match their interests. With thousands of available titles on streaming services and movie databases, manually searching for suitable content has become difficult, time-consuming, and often frustrating. This issue has increased the importance of intelligent recommendation systems that can simplify movie discovery and improve the user experience.

Traditional browsing methods such as genre categories are no longer sufficient for effective content discovery. Genres such as action, comedy, thriller, romance, or drama are too broad and generalized to capture the actual preferences of viewers. For example, the category *thriller* may include psychological suspense films, crime investigation stories, supernatural horror thrillers, or fast-paced action chase movies. Although these films share the same genre label, they can provide entirely different viewing experiences. A user who enjoys a slow and emotionally intense psychological thriller may not be interested in a loud, action-driven thriller. Therefore, relying only on genre classification leads to inaccurate recommendations and limits the user's ability to discover truly relevant content

Another common method used by many platforms is popularity-based recommendation. Popularity charts such as *Top 10 Movies*, *Trending Now*, or *Most Watched This Week* often promote the same well-known and highly viewed titles repeatedly. While these lists may help some users, they do not address individual taste or personal preference. More importantly, popularity-based systems tend to ignore lesser-known or older films that may strongly match a viewer's interests. Many high-quality movies hidden deep within large catalogues remain undiscovered simply because they are not currently trending. As a result, users miss opportunities to explore content that aligns closely with their preferences.

Collaborative filtering is another widely used recommendation technique in commercial platforms. This method analyzes user behavior such as ratings, watch history, clicks, and preferences, then recommends movies based on similarities between users. Although highly effective in large-scale systems, collaborative filtering faces a major limitation known as the **cold-start problem**. When a new platform is launched or a new user joins with no prior activity, the system lacks enough historical data to generate meaningful recommendations. Similarly, newly added movies without ratings or user interactions may not be recommended at all. This creates a challenge for startups, academic projects, and smaller platforms that do not possess massive user data.

The cold-start problem becomes even more critical when developing a standalone movie recommendation system for educational or prototype purposes. Many student projects or independent applications do not have access to millions of user interactions, large-scale rating databases, or expensive third-party APIs. Therefore, building a recommendation

engine that depends on collaborative filtering becomes impractical in such situations. There is a strong need for an alternative approach that can function effectively without requiring user history or external commercial services.

The specific problem addressed in this project is centered on content discovery through movie similarity analysis. Given the title of a movie that a viewer already enjoyed, the system must automatically identify other movies that are thematically, stylistically, and contextually similar. The recommendation process should be based on the intrinsic characteristics of movies themselves, such as plot descriptions, genres, cast members, directors, keywords, and textual metadata. Instead of asking users to rate multiple films or build a profile over time, the system should instantly generate relevant suggestions from a single movie input.

1.3 Objectives

The principal objectives of the CineMatch Movie Recommendation System project are focused on designing an intelligent, efficient, and user-friendly platform for movie discovery. The project aims to combine Natural Language Processing (NLP), Machine Learning, and Web Development technologies to provide accurate movie recommendations based on content similarity. The objectives are explained below:

- To design and implement an NLP-driven content-based movie recommendation engine using TF-IDF vectorisation and cosine similarity on the TMDB 5000 dataset. The primary objective of this project is to build a recommendation engine that suggests movies similar to a selected title by analyzing movie content rather than user history. The system uses the TMDB 5000 dataset, which contains movie metadata such as genres, keywords, cast, crew, and plot summaries. TF-IDF vectorisation converts textual movie information into numerical vectors, while cosine similarity measures the closeness between movies to generate accurate recommendations.

- To build an NLP preprocessing pipeline using NLTK that cleans, tokenises, and removes stop words from combined movie feature strings.

Another important objective is to preprocess raw textual data before applying machine learning algorithms. Using the NLTK library, the system performs tasks such as text cleaning, converting words to lowercase, removing punctuation, tokenizing sentences into words, and eliminating common stop words such as “the,” “is,” and “and.” This preprocessing step improves the quality of the dataset and increases recommendation accuracy.

- To develop a Flask-based web application with live autocomplete search, responsive movie cards, a detail modal, and a trending row.

The project also aims to create an interactive web interface where users can easily search for movies and receive recommendations instantly. Flask is used as the backend framework to connect the recommendation engine with the frontend. Features such as live autocomplete search help users quickly find movie titles, responsive movie cards display recommendations attractively, detail modals show additional movie information, and a trending row highlights popular movies.

- To integrate the OMDb API for real-time retrieval of movie posters, IMDb ratings, full plot summaries, and trailer search links.

To improve the visual appeal and usefulness of the application, the project integrates the OMDb API. This enables the system to fetch real-time movie posters, ratings, plot details, release information, and external trailer links. As a result, users receive a richer and more engaging browsing experience.

- To persist the trained similarity model using Python's pickle module, eliminating retraining overhead on subsequent application launches.

Efficiency is another key objective of the project. Once the recommendation model is trained, it is saved using Python's pickle module. This allows the application to load the trained model directly whenever it starts, avoiding repeated preprocessing and training processes. It reduces startup time and improves system performance.

- To conduct systematic unit, integration, and end-to-end testing and document results with test cases and sample recommendation outputs.

The final objective is to ensure reliability and correctness through proper software testing. Unit testing verifies individual modules, integration testing checks communication between frontend and backend components, and end-to-end testing validates complete system functionality. Test cases and recommendation results are documented to demonstrate the accuracy and effectiveness of the project.

1.4 Scope of the Project

The scope of the Cine Match Movie Recommendation System is focused on developing an efficient and user-friendly content-based recommendation platform for movies. The project is designed to recommend films based on similarity in content features such as genres, keywords, cast members, crew information, and plot descriptions. Instead of depending on user watch history or ratings, the system analyzes movie metadata to identify titles that are thematically and stylistically related. This makes the project suitable for environments where user data is unavailable or privacy concerns are important.

The recommendation engine is built using the TMDB 5000 dataset, which contains information for approximately 4,800 movies. This dataset provides sufficient diversity across genres, languages, release years, and popularity levels, allowing the system to generate meaningful recommendations from a moderately large collection of films. By using this static dataset, the project ensures consistent results, easier model training, and simplified deployment for academic demonstration purposes.

A major scope of the project is the use of content-based filtering techniques supported by Natural Language Processing (NLP). Textual features from multiple movie attributes are combined and processed using techniques such as tokenization, stop-word removal, TF-IDF vectorisation, and cosine similarity. These methods help the system compare movies based on descriptive content rather than external behavioral data. The recommendation logic remains transparent and interpretable, meaning that suggested movies can be explained through shared genres, keywords, themes, or cast members rather than hidden or black-box algorithms.

The project is intentionally limited to movies only. It does not currently include recommendations for TV series, web shows, documentaries, music albums, podcasts, books, or other entertainment categories. Restricting the scope to movies allows the system to maintain focused functionality, reduce data complexity, and achieve better recommendation accuracy within the selected domain.

Another important boundary of the project is that it does not include a user account system, login module, profile management, or persistent user history. Users can access the

application directly, search for a movie title, and receive recommendations instantly without registration. This lightweight design improves accessibility, reduces storage requirements, and avoids privacy issues related to collecting personal data.

From a deployment perspective, the project is developed as a single-page web application using Flask as the backend framework and HTML, CSS, and JavaScript for the frontend interface. During the current academic phase, the system is intended for local execution or small-scale intranet deployment, where users can run the application on a personal computer or campus network. It is optimized for demonstration, testing, and educational purposes rather than heavy commercial traffic.

1.5 Organisation of the Report

This report is systematically organized into seven chapters to provide a clear and structured understanding of the Cine Match Movie
Recommendation System project. Each chapter focuses on a specific phase of the project, beginning with background concepts and ending with conclusions and future enhancements. The arrangement of chapters enables readers to follow the complete development life cycle of the system in a logical manner.

Chapter 1 introduces the project and presents the overall foundation of the study. It includes the background and motivation for developing a movie recommendation system, the problem statement, project objectives, scope of the project, and the organization of the report. This chapter helps readers understand the importance of recommendation systems in modern entertainment platforms and the need for the proposed solution.

Chapter 2 presents a detailed Literature Review related to recommendation systems and content discovery technologies. It discusses the historical evolution of recommendation methods such as collaborative filtering, contentbased filtering, and hybrid recommendation systems. The chapter also explains recent developments in Natural Language Processing (NLP) techniques that are widely used in intelligent recommendation platforms. Existing research papers, methodologies, tools, and comparative studies are reviewed to establish the academic background of the project.

Chapter 3 focuses on System Analysis. It examines the limitations and challenges of existing movie recommendation systems, such as cold-start problems, overdependence on popularity-based suggestions, lack of personalization, and privacy concerns. Based on these observations, the chapter formally defines the proposed system and explains how the Cine

Match solution overcomes the identified issues using content-based filtering and NLP techniques.

Chapter 4 presents the complete System Design of the project. This chapter describes the software architecture, module interactions, and workflow of the recommendation engine. It includes UML diagrams such as use case diagrams, sequence diagrams, activity diagrams, and class diagrams to visually represent the system structure. The chapter also explains database or dataset models, input design, output design, and user interface planning.

Chapter 5 details the Implementation phase of the project. It explains the development environment, programming languages, frameworks, and tools used in building the application. This chapter provides a breakdown of major modules such as dataset preprocessing, NLP pipeline, recommendation logic, Flask backend, API integration, and frontend design. It may also include code explanations and source code walkthroughs for better technical understanding.

Chapter 6 presents Testing, Evaluation, and Results. Different testing methods such as unit testing, integration testing, and end-to-end testing are described in this chapter. It includes test cases, expected outputs, actual outputs, and screenshots of the running application. The effectiveness of the recommendation engine is evaluated through sample movie recommendations and system performance observations.

Chapter 7 concludes the report by summarizing the achievements and outcomes of the project. It highlights how the Cine Match system successfully addresses movie discovery challenges using intelligent recommendation techniques. This chapter also outlines future enhancements such as cloud deployment, user login systems, hybrid recommendation models, mobile applications, and real-time personalization features.

CHAPTER-2

LITERATURE SURVEY

2. LITERATURE SURVEY

2.1 Overview of Recommendation Systems

In the modern digital world, users are continuously exposed to enormous amounts of information and content through online platforms. Whether it is movies, music, books, products, news articles, or social media posts, the volume of available choices can often become overwhelming. This challenge is commonly known as **information overload**, where users struggle to identify the most relevant or interesting options from a large collection. To address this problem, recommendation systems were developed as intelligent information filtering tools that help users discover items they are likely to prefer or find useful.

A **recommendation system** is a software-based decision support mechanism that predicts the interests of users and suggests suitable items based on available data. These systems are widely used in e-commerce websites, streaming services, online learning platforms, and digital libraries. For example, online shopping platforms recommend products based

on browsing behavior, while streaming services recommend movies or songs according to user interests. By reducing search effort and presenting personalized suggestions, recommendation systems improve user satisfaction and increase platform engagement.

The formal study of recommendation systems began during the mid-1990s, when researchers recognized the growing need for automated filtering methods in online environments. One of the earliest and most influential projects in this field was GroupLens Research at the University of Minnesota. The GroupLens project introduced the concept of **collaborative filtering** for recommending Usenet news articles. Instead of relying on content alone, the system analyzed ratings and opinions of multiple users to identify patterns of shared interests. This innovation became the foundation for many modern recommendation engines.

As the internet expanded and digital platforms gained millions of users, recommendation systems evolved rapidly. Commercial companies realized that intelligent recommendations could significantly increase sales, customer retention, and user engagement. A major milestone in the history of recommendation research occurred in 2006 when Netflix announced the famous **Netflix Prize**, offering one million dollars to any team that could improve the accuracy of its movie recommendation algorithm by 10 percent. This competition attracted researchers, data scientists, and engineers from around the world. It led to major advancements in machine learning, collaborative filtering, matrix factorization, and ensemble methods. The Netflix Prize demonstrated both the business value and academic importance of recommendation technology.

2.2 Content-Based Filtering

Content-based filtering is a recommendation technique in which items are suggested to users by examining the attributes and properties of the items themselves. Unlike collaborative filtering, which depends on the preferences and behavior of many users, content-based filtering focuses entirely on the content or metadata associated with each item. In the context of movies, this may include features such as genre, storyline, keywords, cast members, director, language, production company, release year, and audience category. The main objective is to identify items that share similar characteristics with those already liked by the user.

The functioning of content-based filtering usually begins by creating a profile of each item. This profile contains all important descriptive features related to the item. For movies, textbased information such as plot summaries and keywords can be converted into

numerical form using techniques like TF-IDF vectorisation or word embeddings. Once each movie is represented mathematically, similarity measures such as cosine similarity are used to compare one movie with others in the dataset. Titles with the highest similarity scores are then recommended to the user.

One of the greatest advantages of content-based filtering is that it performs well even when user history is limited or unavailable. A new user can receive recommendations simply by selecting one movie they like, without needing to rate many titles first. This helps solve the cold-start problem commonly faced by collaborative filtering systems. It is also useful for newly added movies, because recommendations depend on metadata rather than waiting for ratings or user interactions.

Another important benefit is privacy. Since content-based systems rely mainly on item information rather than personal activity logs, they reduce the need for collecting sensitive user behavior data. This makes them suitable for privacy-conscious applications. In addition, recommendations are more transparent and interpretable. For instance, the system can explain that a movie was recommended because it shares the same genre, lead actor, director, or plot theme as a previously liked film.

2.3 Collaborative Filtering

Collaborative filtering is one of the most popular and widely adopted recommendation techniques used in modern digital platforms. It is based on the idea that users who have shown similar interests in the past are likely to have similar preferences in the future. Instead of analyzing the content of items such as movie genres or product descriptions, collaborative filtering studies the behavior and choices of users. If two people liked many of the same movies, songs, books, or products, the system assumes that they may also like other items preferred by one another.

The main strength of collaborative filtering is that it does not require deep knowledge of item content. It can recommend items even when metadata such as genre, description, or keywords is limited. This makes it highly suitable for products, music, or content where descriptive information may be incomplete. It is also effective in identifying surprising or unexpected recommendations, since it is based on real user behavior rather than obvious feature similarity. Collaborative filtering generally depends on user interaction data such as ratings, likes, clicks, watch history, purchase history, browsing behavior, or time spent on content. These interactions are stored in a user-item matrix where rows represent users and

columns represent items. The values in the matrix indicate whether a user liked, viewed, rated, or interacted with an item. Recommendation algorithms then analyze patterns within this matrix to predict unseen preferences.

There are two major approaches in collaborative filtering:

User-Based Collaborative Filtering

In user-based collaborative filtering, the system identifies users with tastes similar to the target user and recommends items liked by those similar users. For instance, if several users with matching preferences enjoyed a particular movie that the target user has not yet seen, the system may recommend that movie. Similarity between users is often measured using methods such as cosine similarity, Pearson correlation, or Euclidean distance.

Item-Based Collaborative Filtering

In item-based collaborative filtering, the focus shifts from users to items. The system identifies items that received similar responses from many users. If people who liked one movie also consistently liked another movie, those two titles are considered similar. If a user enjoyed one of them, the other can be recommended. This method is widely used in commercial systems because item relationships often remain more stable than user relationships.

2.4 Hybrid Approaches

Hybrid recommendation systems are advanced recommendation models that combine two or more recommendation techniques, most commonly **collaborative filtering** and **contentbased filtering**, in order to overcome the limitations of each individual method. Since no single recommendation technique performs perfectly in every situation, hybrid systems are designed to take advantage of the strengths of multiple approaches while minimizing their weaknesses. As a result, hybrid recommendation systems are widely used in modern commercial platforms such as Netflix, Amazon, YouTube, and Spotify.

The main motivation behind hybrid systems is that collaborative filtering and contentbased filtering solve different types of problems. Collaborative filtering is excellent at identifying hidden user preferences through collective behavior, but it struggles with new users, new items, and sparse datasets. Content-based filtering works well when metadata is available

and does not depend on user history, but it may generate overly narrow or repetitive recommendations. By combining both approaches, a hybrid system can provide more balanced, accurate, and diverse suggestions.

For example, consider a movie streaming platform. If a user has watched and rated many films, the system may use collaborative filtering to recommend titles liked by similar users. However, if the same user is new and has no viewing history, the platform may rely on content-based filtering by analyzing the genre, cast, keywords, or storyline of a selected movie. In this way, hybrid systems adapt intelligently according to available data.

2.5 NLP Techniques in Recommendation

Natural Language Processing (NLP) has become one of the most important technologies in modern recommendation systems, especially in content-based recommendation models. Many digital platforms store large amounts of textual information such as movie descriptions, reviews, genres, cast details, keywords, and summaries. Traditional recommendation methods often rely only on numerical ratings or user behavior, but NLP enables systems to understand and analyze the meaning of textual data. This makes recommendations more accurate, interpretable, and useful, particularly when user interaction data is limited.

In movie recommendation systems, textual metadata contains valuable information about the nature of each film. A plot summary may reveal whether a movie is romantic, suspenseful, futuristic, emotional, or comedic. Cast names indicate actors associated with certain styles or fan preferences. Directors may influence storytelling patterns, cinematography, or genre tendencies. Keywords such as “space travel,” “revenge,” “superhero,” or “mystery” help identify thematic similarities. NLP techniques transform this unstructured text into structured numerical representations that can be compared mathematically.

One of the most influential and widely used NLP methods in information retrieval and recommendation is TF-IDF (Term Frequency–Inverse Document Frequency), introduced and formalized by researchers such as Gerard Salton and Chris Buckley. TF-IDF is a statistical weighting technique used to measure how important a word is within a document relative to a collection of documents.

The Term Frequency (TF) component measures how often a word appears in a specific document. If a term appears multiple times in a movie description, it is assumed to be more relevant to that movie. For example, if the word “detective” appears several times in a plot summary, it likely reflects a core theme of the film.

2.6 Summary Table

Authors / Year	Technique	Relevance to CineMatch
Resnicktal, 1994	Collaborative Filtering	Historical baseline; rejected for cold-start limitation

Pazzani&Billsus, 2007	TF-IDF content profiles	Direct basis for vectorisation strategy
Lops et al., 2011	Feature engineering review	Motivated stopword removal and NLP preprocessing

Table 2.1

CHAPTER - 3

SYSTEM ANALYSIS

3. SYSTEM ANALYSIS

3.1 Survey of Existing Tools

Before designing and implementing the proposed system, it was important to study the tools and platforms already available in the domain of movie discovery, streaming analytics, and recommendation services. A survey of existing tools helps identify what features are already solved effectively, what limitations still exist, and where a new system can provide value. It also prevents unnecessary redevelopment of functions that are already widely available. For this project, the review focused on three major categories of tools: consumer-facing content discovery platforms, analytical dashboards, and academic notebook solutions.

The purpose of this survey was to understand what the Cine Match project needed to build, and equally important, what it could intentionally avoid building. Many platforms already solve narrow problems such as title search or watchlist management, but fewer systems provide transparent, content-based recommendation features using open datasets and deployable web applications.

Another relevant category includes movie community platforms such as Letterboxd and review ecosystems connected to IMDb.

These platforms provide rich user-generated data including:

- Ratings
- Reviews
- Watchlists
- Popular lists
- Favorite films
- Community trends

For film enthusiasts, these tools are highly valuable because they support discussion, social engagement, and discovery through curated lists. Users can explore films based on public ratings and community taste.

3.2 Identified Limitations of Existing Approaches

Limitation	Impact	How This Project Addresses It
No cross-platform unified view	Users must visit multiple sites for comparison	Single dashboard compares all 4 platforms simultaneously
Static notebook outputs	Findings cannot be queried interactively	Flask routes enable dynamic data access
No ML clustering in consumer tools	Pattern discovery requires user-side analysis	K-Means runs server-side; results served via API
No REST API for catalogue data	Cannot integrate streaming data into other apps	/api/stats, /api/clusters, /api/movies endpoints

Non-replaceable datasets	Tools are locked to specific data snapshots	Drop-in CSV replacement triggers automatic retraining
Paid tiers for advanced analytics	Students and researchers face cost barriers	Fully open-source, zero-cost to run locally

Table 3.1 Limitations of Existing Approaches

3.3 Proposed System Overview

The proposed system is a self-contained analytical web application developed to provide intelligent movie recommendations and useful data insights through a simple and efficient architecture. The application is designed to function independently without requiring complex cloud infrastructure or external database servers. It combines data processing, machine learning, and interactive web presentation into one integrated system that can be deployed locally or within an intranet environment. The main objective of the system is to offer fast, accurate, and user-friendly movie recommendations while maintaining a lightweight and scalable design.

The system is organized into three major architectural layers: the data layer, the machine learning layer, and the presentation layer. The data layer is responsible for storing and managing the movie dataset in CSV format. This dataset contains information such as movie titles, genres, cast members, crew details, ratings, popularity, release dates, and plot summaries. The CSV file is loaded using Pandas, which is used to clean missing values, remove duplicate entries, standardize columns, and prepare the data for analysis. By using a CSV-based structure, the project remains portable, easy to maintain, and suitable for academic development.

The machine learning layer is built using Scikit-learn and provides the analytical intelligence of the system. This layer performs tasks such as feature extraction, clustering, similarity analysis, and recommendation generation. Depending on the implementation, algorithms such as TF-IDF vectorisation, cosine similarity, and K-Means clustering are used to identify relationships between movies. By converting textual and numerical features into machinereadable vectors, the system can automatically group similar movies and generate meaningful recommendations for users.

The presentation layer is implemented using Flask for backend routing and Jinja2 for rendering dynamic HTML pages. The frontend interface is enhanced using Chart.js to display charts, statistics, and interactive visual elements. Through this layer, users can search for movies, view recommended titles, explore trending content, and access analytical dashboards through a modern browser interface.

3.4 Comparative Feature Analysis

Feature	**justwatch**	**Kaggle Notebooks**	**This Project**
Cross-platform view	partial	Yes(static)	**Yes(interactive)**
k-Means clustering	NO	Yes(static)	Yes(live API)
REST API access	Paid only	NO	Yes(free)
Replaceable dataset	NO	Manual	Automatic retraining
Open source	NO	**Yes**	**yes**
Local deployment	NO	**Yes**	**yes**
Real-time silhouette score	NO	No	**yes**

Table 3.2 Comparative Feature Analysis

CHAPTER-4 SYSTEM REQUIREMENTS

4. SYSTEM REQUIREMENTS

4.1 FUNCTIONAL REQUIREMENTS

Functional requirements are the specific features and services that the proposed system must provide to the end user. These requirements describe the core operations that the system should perform and define how users interact with the application. They represent the visible and practical capabilities of the software, unlike non-functional requirements which focus on performance, security, or reliability. Functional requirements are essential because they act as a contract between users, developers, and stakeholders, ensuring that the final product satisfies the intended objectives.

In software engineering, functional requirements clearly specify the inputs given to the system, the processing or operations performed by the system, and the outputs expected as results. They explain what the system should do under normal and special conditions. These requirements help developers design modules, write code, test functionality, and validate whether the completed system meets user expectations. For the **Cine Match Movie Recommendation System**, functional requirements focus on enabling users to search movies, receive recommendations, view details, and interact with the web application smoothly.

The system must provide a **movie search facility** where users can enter the name of a movie title. The search function should support quick retrieval of matching movie names and may include autocomplete suggestions to improve usability. This helps users easily locate their desired movie without manually browsing a large dataset.

The system must generate **movie recommendations** based on the selected movie title. When a user chooses a movie, the recommendation engine should analyze similarity scores and return a list of the most relevant movies. These recommendations should be generated automatically using content-based filtering techniques such as TF-IDF and cosine similarity.

The application must display recommendation results in a user-friendly format using movie cards or list views. Each recommended movie should include important details such as title, poster image, release year, rating, genre, or short description. Presenting information visually improves user engagement and decision-making.

4.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements are the standards and quality attributes that define how a software system should operate. Unlike functional requirements, which describe the specific features or services provided by the system, non-functional requirements focus on the performance, reliability, usability, and overall quality of those features. In simple terms, they describe how well the system should perform its tasks rather than what tasks it performs. These requirements are extremely important because even a feature-rich system may fail if it is slow, insecure, difficult to use, or unstable.

Non-functional requirements help ensure that the software functions efficiently, securely, and consistently under different operating conditions. They establish measurable expectations related to response time, accuracy, maintainability, scalability, and system availability. These

requirements are often considered quality constraints because they influence user satisfaction and long-term success of the application.

For a deepfake detection system leveraging deep learning, non-functional requirements become even more critical. Such systems process images, videos, or audio data using complex neural network models that require significant computational resources. Therefore, the software must be optimized to handle large data volumes while maintaining speed and prediction accuracy. Since deepfake detection is often used in sensitive environments such as media verification, cybersecurity, law enforcement, and social platforms, the system must also provide dependable and trustworthy results.

One major non-functional requirement is performance. The deepfake detection system should analyze media files quickly and produce predictions within an acceptable time frame. Users expect near real-time detection when uploading suspicious content. Efficient model inference, GPU acceleration, and optimized preprocessing pipelines help achieve faster execution.

the following non-functional requirements must be met:

- Performance and Speed
- Scalability
- Availability and Reliability
- Security

4.3 HARDWARE REQUIREMENTS

Component	Minimum Specification	Recommended Specification
Processor	Dual-core x86-64 (e.g. Intel Core i3)	Quad-core x86-64 (Core i5 or better)
RAM	4 GB	8 GB (especially for larger CSVs)
Storage	500 MB free	2 GB free (for virtual environment)
Display	1280 × 720 resolution	1920 × 1080 for optimal chart display

Network	Not required at runtime	Required for initial pip install

Table 4.1 HARDWARE REQUIREMENTS

4.3 SOFTWARE REQUIREMENTS

Package	Version	Purpose
Python	3.9+	Core runtime environment
Flask	>=2.3.0	Web framework — routing, templating, dev server
Pandas	>=1.5.0	DataFrame operations, CSV I/O, aggregation

NumPy	>=1. 23.0	Array operations, NaN handling
Scikitlearn	>=1. 2.0	Standard KMeans, Scaler,silhouette_score
Matplotlib	>=3. 6.0	Static chart generation (notebooks)
Seaborn	>=0. 12.0	Statistical visualization (heatmap, distributions)
Jinja2	bundled with Flask	HTML templating
Werkzeug	bundled with Flask	WSGI utilities

Table 4.2 SOFTWARE REQUIREMENTS

CHAPTER-5 SYSTEM STUDY 5 . SYSTEM STUDY

5.1 FEASIBILITY STUDY

The feasibility study is an important stage in the system development life cycle where the practicality and viability of the proposed project are carefully examined before implementation begins. In this phase, the project is analyzed from different perspectives to determine whether it can be developed successfully using available resources, time, technology, and budget. A general business proposal is also prepared along with an initial development plan and estimated costs. The main purpose of conducting a feasibility study is to ensure that the proposed system does not become a burden to the organization in terms of financial cost, technical difficulty, legal issues, or operational complexity.

During system analysis, feasibility study helps developers and stakeholders understand whether the project is realistic and beneficial. It reduces the risk of project failure by identifying possible challenges in advance. For the Cine Match Movie Recommendation System, feasibility analysis confirms that the project is suitable for academic implementation and small-scale deployment because it uses open-source technologies, a structured dataset, and lightweight architecture.

For proper feasibility analysis, it is essential to understand the major system requirements such as dataset availability, software tools, hardware support, user needs, development time, and maintenance effort. Based on these considerations, the project can be evaluated under the following categories:

- Technical Feasibility
- Operational Feasibility
- Economic Feasibility
- Legal and Ethical Feasibility

TECHNICAL FEASIBILITY

- Technical feasibility determines whether the required hardware, software, tools, and technical skills are available to build the proposed system. In the Cine Match project, the system is developed using Python, Flask, Scikit-learn, Pandas, and NLP libraries.

- The project does not require high-end servers or expensive infrastructure. A normal computer with moderate RAM and storage is sufficient to run the application. Since the technical requirements are easily available and manageable, the project is technically feasible.

.OPERATIONAL FEASIBILITY

- Operational feasibility evaluates whether the proposed system can function effectively in the intended environment and whether users can operate it successfully. The Cine Match system is designed with a simple and interactive web interface where users can search for a movie title and receive recommendations instantly.

- The system does not require special training or technical expertise, making it easy for students, staff, and general users to operate. It also reduces manual effort in finding suitable movies by automating the recommendation process. Since the system is userfriendly, practical, and easy to maintain, it is operationally feasible.

ECONOMICAL FEASIBILITY

- Economic feasibility examines whether the benefits of the project justify the development and maintenance cost. The Cine Match project is highly cost-effective because it uses opensource software tools and free movie datasets. There is no need for expensive licenses, commercial software, or dedicated enterprise hardware.

- The development cost is limited mainly to time and basic computing resources. At the same time, the system provides strong academic value, practical learning experience, and a fully working recommendation platform. Since the project delivers useful outcomes with minimal investment, it is economically feasible.

LEAGL AND ETHICAL FEASIBILITY

- Legal and ethical feasibility ensures that the system complies with laws, software licenses, and responsible data practices. The Cine Match project uses publicly available datasets and open-source libraries that are legally permitted for educational and development purposes.

- The system does not store sensitive personal information such as passwords, payment details, or private user history. This reduces privacy concerns and legal risk. Ethically, the recommendation engine is transparent because it uses interpretable content-based logic rather than hidden profiling methods.

5.2 FEASIBILTY ANALYSIS

Feasibility analysis is an important process carried out during system planning and development to determine whether the proposed project is practical, beneficial, and achievable. It helps decision-makers understand whether the system can be developed successfully within available budget, technical resources, and user acceptance levels. Conducting feasibility analysis before implementation reduces risks, prevents unnecessary expenditure, and ensures that the project provides real value to the organization.

For the Cine Match Movie Recommendation System, feasibility analysis confirms that the project is suitable for academic and small-scale practical deployment. The system uses modern but lightweight technologies, open-source tools, and a simple operational structure. Three major considerations involved in feasibility analysis are economical feasibility, technical feasibility, and social feasibility

Three key considerations involved in the feasibility analysis are

- Economical feasibility
- Technical feasibility
- Social feasibility

ECONOMICAL FEASIBILITY

- Economical feasibility examines the financial impact of the proposed system on the organization. It determines whether the project can be completed within the available budget and whether the expected benefits justify the cost of development, implementation, and maintenance.

- Every organization has limited funds that can be allocated to research and development activities. Therefore, all expenditures must be carefully justified. In the case of Cine Match, the overall cost of development is very low because most of the required technologies are freely available. The project uses open-source tools such as Python, Flask, Scikit-learn, Pandas, HTML, CSS, and JavaScript.

- The movie dataset used for training and testing is also publicly available, reducing the need for expensive data acquisition. Since the project can run on a standard personal computer without requiring specialized servers, infrastructure costs are minimal. Only optional customized tools, premium APIs, or future deployment services may involve cost.

- Thus, the developed system remains well within budget limits while still delivering useful functionality and educational value. Therefore, the project is economically feasible.

TECHNICAL FEASIBILITY

Technical feasibility is conducted to determine whether the organization has the necessary hardware, software, and technical expertise to develop and run the proposed system. A system should not place excessive demands on available technical resources, as this can create implementation difficulties and increase operational burden.

For Cine Match, the technical requirements are modest and practical. The application can be developed using common programming environments and standard computers or laptops. It does not require high-performance enterprise infrastructure or advanced proprietary software.

The recommendation engine is based on proven technologies such as:

- Content-based filtering
- TF-IDF vectorisation
- Cosine similarity
- CSV dataset processing
- Web application development using Flask

These technologies are stable, well-documented, and supported by large developer communities. The system can run efficiently with moderate RAM and storage. Only minimal or no changes are required to the client environment for implementation.

Because the project uses readily available technical resources and manageable computing power, it is technically feasible.

SOCIAL FEASIBILITY

Social feasibility refers to the level of acceptance of the proposed system by its intended users. Even a technically strong system may fail if users are unwilling to adopt it. Therefore, it is important to evaluate how users perceive the system and whether they are comfortable using it.

For the Cine Match project, the user interaction model is simple and friendly. Users only need to enter a movie title and receive recommendations through a visually attractive interface. No technical knowledge is required to use the system. This simplicity increases acceptance among students, faculty members, and general users.

Training requirements are minimal because the system is intuitive. However, proper demonstrations, guidance, and documentation can further improve user confidence. Users should not feel threatened by automation or machine learning recommendations. Instead, they should view the system as a helpful assistant that saves time and improves movie discovery.

User feedback and constructive criticism are also valuable. Since end users interact directly with the application, their suggestions can help improve search quality, recommendation relevance, and interface design.

CHAPTER-6 SYSTEM DESIGN

6.1 System Architecture

The system architecture of the **Movie Recommendation System** is designed using a modular and layered approach that ensures efficient communication between the user interface, backend application, machine learning engine, and external data services. This structured architecture helps the system operate smoothly, improves maintainability, and allows future enhancements without affecting the complete application. Each layer performs a specific responsibility, making the system organized, scalable, and easy to manage.

The architecture mainly consists of four important components: the **Frontend Layer**, **Backend Layer**, **Machine Learning Layer**, and **External API Layer**. These components work together to receive user input, process recommendations, and display results in an interactive format.

The **Frontend Layer** is developed using HTML, CSS, and JavaScript. This layer acts as the user interface of the application. It allows users to search for movie titles, view recommended movies, and interact with the system through a clean and responsive webpage. JavaScript is used to provide dynamic features such as autocomplete search suggestions, loading effects, popup modals, and smooth content updates without reloading the page.
When a user enters a movie title and submits the request, the input is sent to the **Backend Layer**, which is implemented using Flask. Flask acts as the central controller of the system and manages communication between all components. It receives search requests from the frontend, validates user input, calls the recommendation engine, fetches external movie details, and sends final results back to the browser. Flask routes are used to manage pages such as homepage, recommendation results, movie detail endpoints, and API responses. The core intelligence of the application lies in the **Machine Learning Layer**. This layer is

responsible for analyzing the dataset and generating recommendations. The movie dataset contains titles, genres, cast, crew, keywords, plot overviews, and other metadata. Before generating recommendations, the raw data is preprocessed using libraries such as Pandas and NLP tools. Textual features are cleaned, combined, tokenized, and converted into numerical form using **TF-IDF vectorisation**.

Once movie descriptions are transformed into vectors, **cosine similarity** is used to measure similarity between movies. When a user selects a movie, the system compares that movie's vector with all others in the dataset and identifies the top most similar movies based on content relevance. These recommended titles are then ranked and returned to the backend for presentation.

To improve the quality and visual attractiveness of results, the system integrates with the **OMDb API**, an external movie information service. Through this API, the application can fetch real-time movie posters, IMDb ratings, release year, runtime, genre details, and plot summaries. This enriches the recommendation output and gives users a professional browsing experience similar to commercial streaming platforms.

After processing is complete, the backend sends the final response back to the frontend. Recommended movies are displayed as cards containing posters, titles, ratings, and quick details. Users can click on any movie to open a modal or detailed view for more information. This creates an interactive and user-friendly recommendation environment.

Fig:6.1 System Architecture

6.2 UML Diagrams

Unified Modeling Language (UML) diagrams are an essential part of software engineering and system design. They are used to visually represent the structure, behavior, workflow, and interaction of a system from multiple perspectives. UML provides a standardized modeling language that helps developers, designers, testers, and stakeholders understand how the system is organized, how different modules are connected, and how data flows between components. By using UML diagrams, complex software systems can be explained in a clear, simple, and professional manner.

Unified Modeling Language (UML) diagrams are an essential part of software engineering and system design. They are used to visually represent the structure, behavior, workflow, and interaction of a system from multiple perspectives. UML provides a standardized modeling language that helps developers, designers, testers, and stakeholders understand how the system is organized, how different modules are connected, and how data flows between components. By using UML diagrams, complex software systems can be explained in a clear, simple, and professional manner.

UML diagrams are highly useful during the **system design phase** because they allow developers to visualize the project before coding begins. They help identify system requirements, module relationships, and process flow early in development. This reduces

design errors and saves implementation time. For example, a use case diagram helps define user actions, while a class diagram helps design software classes and their relationships.

During the **implementation phase**, UML diagrams act as a blueprint for developers. Programmers can refer to diagrams to understand module responsibilities, communication flow, and logic structure. This improves coding consistency and teamwork, especially when multiple developers are involved.

UML also improves communication between technical and non-technical stakeholders. Project guides, faculty members, clients, or management teams may not understand source code, but they can easily understand diagrams. This makes UML highly valuable in academic projects and professional software development.

The following UML diagrams model the system from multiple perspectives.

6.2.1 Use Case Diagram

A Use Case Diagram is one of the most important UML diagrams used to represent the functional behavior of a system from the user's perspective. It identifies the different actors who interact with the system and the services or functionalities provided to them. Use case diagrams help developers understand system requirements clearly by showing what the system should do rather than how it is internally implemented. They are especially useful during the requirement analysis and system design phases.

The primary actor is the **Visitor** (any user of the web application). A secondary actor is the **OMDb API** (external system). The system exposes five main use cases:

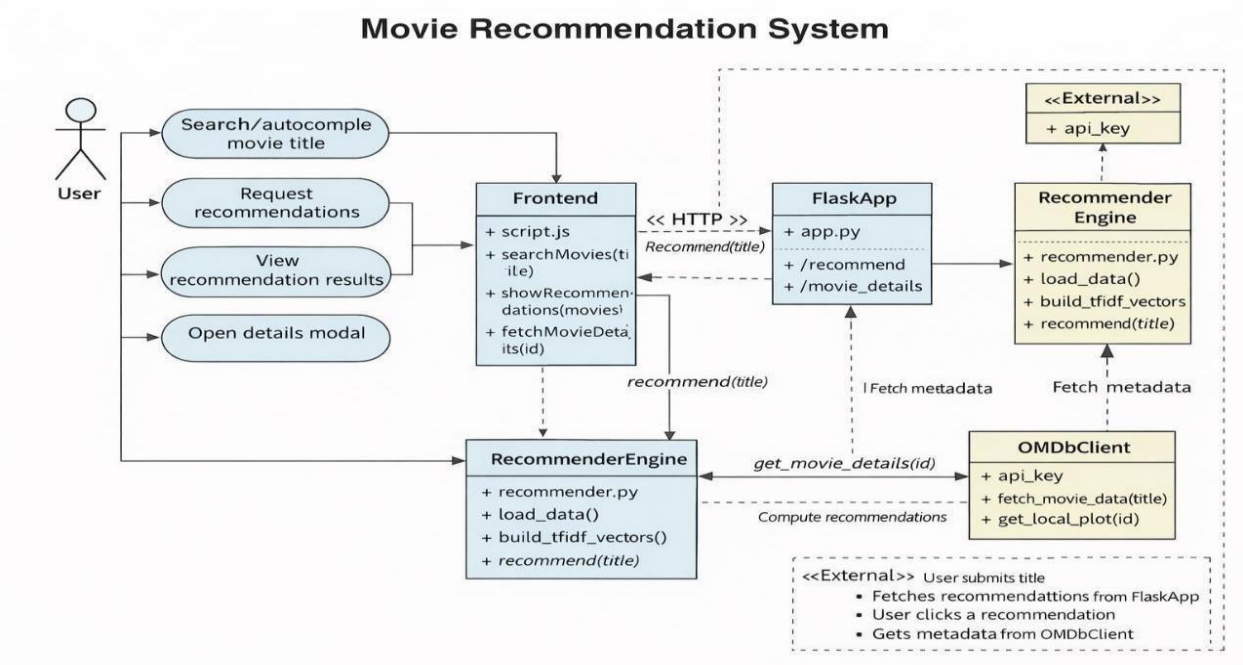


Fig:6.2 UML Diagrams

- **Search Movie:** The visitor types a title into the search bar and submits.
- **Use Autocomplete:** The system returns matching titles as the visitor types (minimum 2 characters).
- **View Recommendations:** The system returns the ten most similar films with posters and scores.
- **View Movie Details:** The visitor clicks a card to open the detail modal with full metadata.
- **Browse Trending:** The homepage displays six pre-selected popular titles with posters and ratings.

6.3 Class Diagram

A Class Diagram is one of the most important UML structural diagrams used to represent the static structure of a software system. It describes the classes present in the application, their attributes, methods, and the relationships between them. Class diagrams help developers understand how the system is organized internally and how different components interact with each other. They are widely used during system design because they provide a blueprint for implementing the software in an object-oriented manner.

The four principal classes and their responsibilities are:

- **FlaskApp:** Manages HTTP routing, global model state (df_movies, similarity), and dependency wiring. Attributes: OMDB_API_KEY, MODEL_PATH. Methods: get_ model(), index(), get _recommendations (), movie _details(), Search _suggestions().
- **Recommender:** Owns the ML pipeline. Methods: load_and_preprocess(), build_ similarity(), save _model(), load_ model(), recommend(). Returns MovieDTO lists.
- **MovieDTO:** Data transfer object. Attributes: title, score, poster, year, genre, director, actors, plot, imdb_rating, trailer_url.

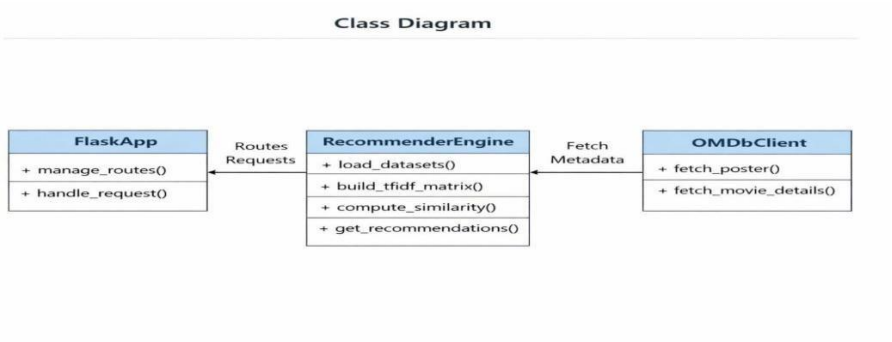


Fig:6.3 Class Diagram

6.4 Sequence Diagram

The sequence of interactions for a standard recommendation query is: (1) User enters a title and clicks Search. (2) Browser fires a POST /recommend request with JSON body. (3) FlaskApp calls get_model() — returns cached (df, similarity) from memory. (4) FlaskApp calls Recommender.recommend(title, df, similarity, top_n=10). (5) recommend() performs case-insensitive partial match, retrieves row index, sorts similarity[index] descending, returns top-10 as list of dicts. (6) FlaskApp iterates results and calls OmdbClient.fetch_poster(title) for each. (7) OMDb returns poster URLs. (8) FlaskApp assembles final JSON response. (9) Browser renders 10 movie cards in the grid.

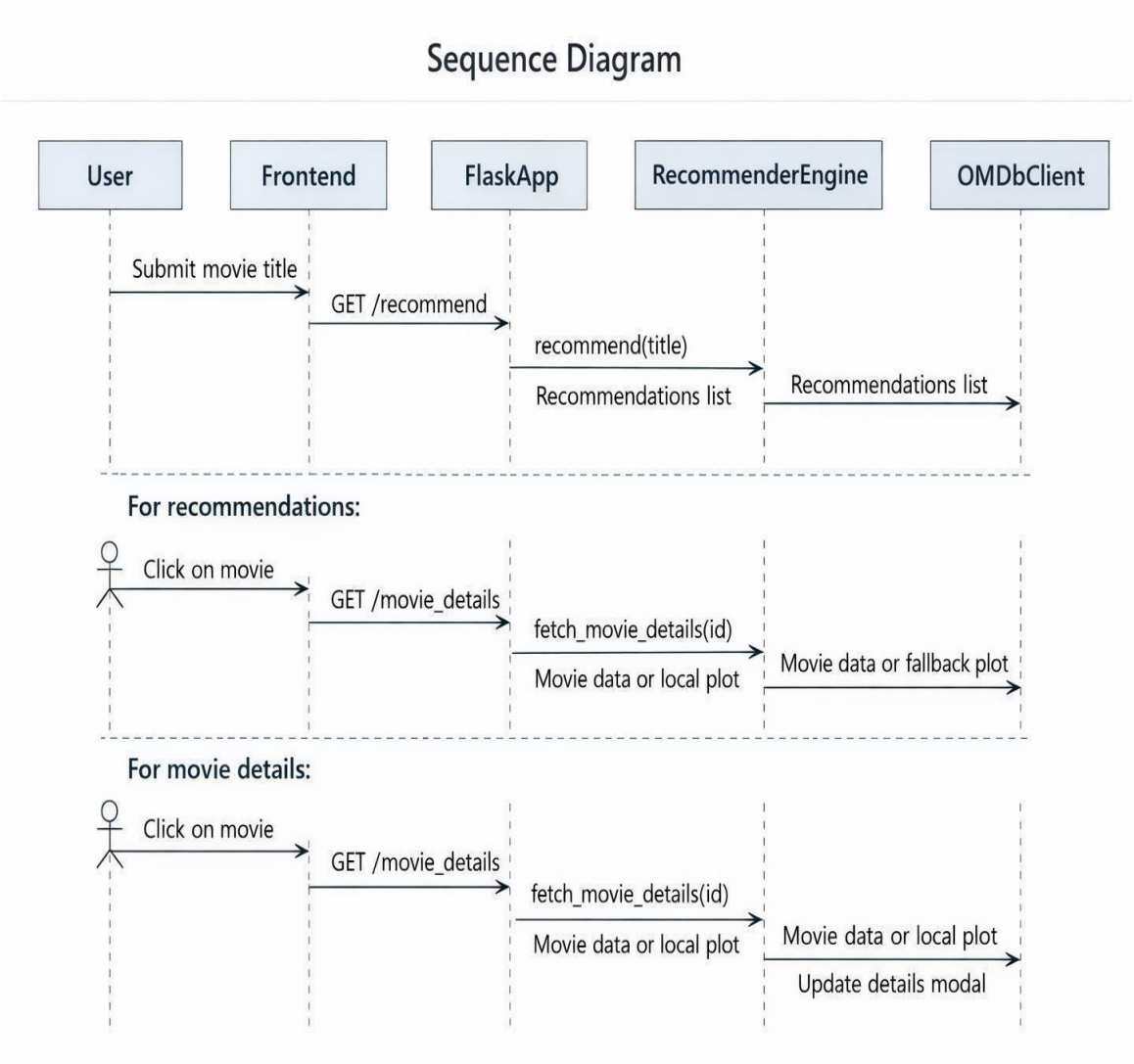


Fig:6.4 Sequence Diagram

6.5 Activity Diagram

On application startup, Flask executes get_model(). The activity flow is: Check whether model/similarity.pkl exists on disk. If yes, deserialise (df_movies, similarity) and load into global state — fast path, approximately 1 second. If no, execute load_and_preprocess() to read CSVs, merge on film id, extract features, build tags, apply NLP preprocessing. Then execute build_similarity() to vectorise with TF-IDF and compute the cosine similarity matrix. Then execute save_model() to serialise both objects to similarity.pkl and load into global state. All subsequent calls to get_model() within the same server session return the cached in-memory objects immediately.

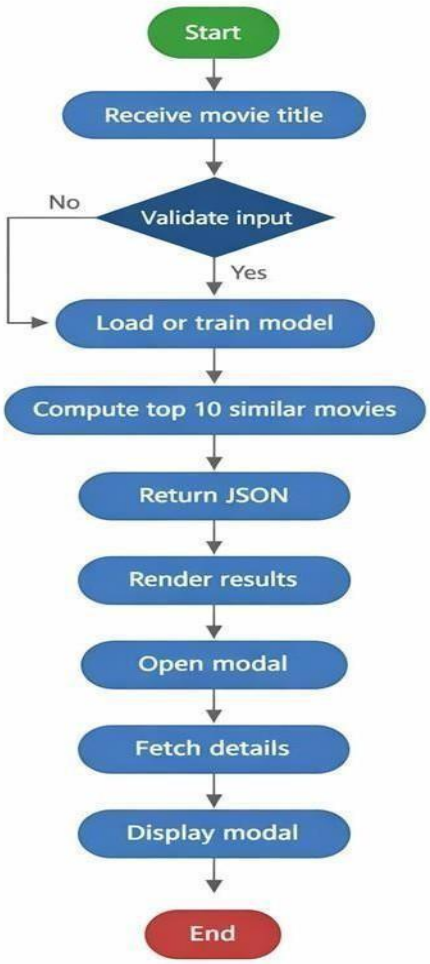


Fig:6.5 Activity Diagram

6.6 Colloboration Diagram

The data model is intentionally simple. There is one primary entity — **Movie** — with attributes: id (PK, integer), title (string), overview (text), genres (list), keywords (list), cast (list, max 5), crew (director name), tags (derived). There is no User entity, no Ratings table, and no Transaction log. The system is stateless by design. The similarity matrix is stored externally as a serialised NumPy array and is not represented in a relational schema.

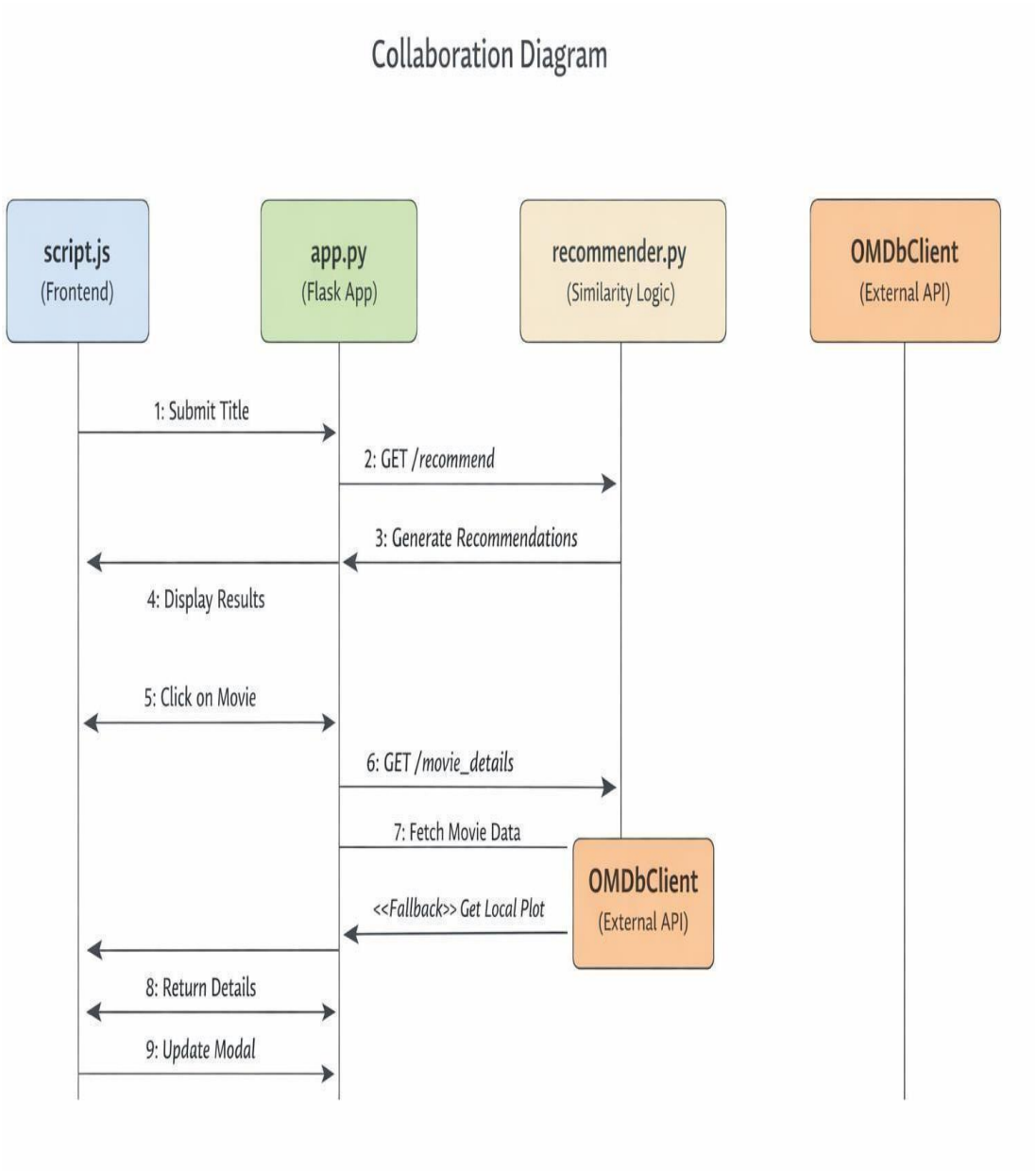


Fig:6.6 **Colloboration Diagram**

CHAPTER-7 INPUTS AND OUTPUTS

7.1 INPUT DESIGN

Input design is one of the most important aspects of system design because it acts as the primary link between the information system and the user. It involves developing methods, specifications, and procedures for collecting data and converting it into a usable form for system processing. Proper input design ensures that accurate data enters the system, reduces processing errors, improves efficiency, and enhances the overall user experience. Since every output depends on the quality of input provided, careful planning of input methods is essential for the success of any software application.

Input data may be entered into the system in different ways. It can be captured from written or printed documents, imported from files, or directly entered by users through keyboards,

forms, or digital interfaces. In modern web-based applications, users commonly provide input through search boxes, dropdown lists, buttons, and online forms. The design of input should make data entry simple, fast, and secure while minimizing user effort.

For the Movie Recommendation System, input design focuses mainly on allowing users to enter movie titles, search queries, and navigation requests through an easy-to-use web interface. Since the system is intended for general users, the input process must be intuitive and error-free. A well-designed input module helps users quickly search for movies and receive recommendations without confusion or unnecessary steps.

The major objective of input design is to control the amount of input required, minimize typing effort, avoid duplicate data entry, reduce delays, and prevent invalid data from entering the system. It also ensures privacy and security by accepting only necessary information and handling user requests safely.

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy.

Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

7.1.1 OBJECTIVES

1. Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is very important because it helps to avoid errors during

the data entry process and ensures that accurate information is provided to management for decision-making through the computerized system.

2. The main objective of input design is to create simple, user-friendly, and efficient data entry screens that can handle a large volume of data without confusion. Properly designed screens reduce the time required for entering data and improve the overall performance of the system.
3. Input design aims to make data entry easier, faster, and free from mistakes. The forms and screens are arranged in such a way that users can easily understand the required fields and enter the correct details in the proper format.
4. Another objective of input design is to provide validation checks whenever data is entered. The system verifies the correctness of data such as mandatory fields, numeric values, date formats, duplicate entries, and invalid records. This improves data accuracy and reliability.
5. Appropriate messages, warnings, and error notifications are displayed whenever necessary so that users can immediately correct mistakes. This prevents confusion and helps users operate the system smoothly.
6. Input design also provides record viewing, editing, updating, and deleting facilities whenever required. This helps users manage records effectively and maintain updated information in the database.
7. A good input design ensures security by allowing only authorized users to enter or modify data. This protects sensitive information from unauthorized access.
8. It also helps in reducing manual paperwork and maintaining digital records in an organized manner. This saves storage space, time, and effort.
9. Another objective is to create an input layout that is easy to learn, simple to follow, and convenient for all users. Even a new user should be able to use the system without much training.

7.2 OUTPUT DESIGN

A quality output is one that meets the requirements of the end user and presents the information in a clear, accurate, and understandable manner. In any system, the results of processing are communicated to users and to other systems through outputs. In output design, it is determined how the information is to be displayed for immediate use on the screen as well as in the form of printed hard copies. Output is the most important and direct source of information to the user. Efficient and intelligent output design improves the relationship between the system and the user and supports effective decision-making.

1. Designing computer output should proceed in an organized and well-planned manner. The right output must be developed while ensuring that each output element is designed so that users can understand and use the system easily and effectively.
2. During output design, analysts should identify the specific outputs needed to meet user requirements. This includes understanding what information is required, when it is required, and in what format it should be presented.

3. Appropriate methods for presenting information should be selected. Outputs may be displayed on screens, printed as reports, sent through emails, or stored digitally depending on the needs of the organization

4. Output design should ensure that information is presented clearly with proper headings, labels, tables, charts, and formatting so that users can interpret the results quickly and accurately.
5. Reports, documents, summaries, invoices, receipts, and other formats containing information produced by the system should be created in a user-friendly manner 6. The output should be accurate, complete, relevant, and delivered on time so that users can make correct decisions without delay.

   7. Good output design reduces confusion, saves time, and improves user satisfaction by presenting only meaningful and necessary information.

   8. Security should also be considered in output design so that confidential information is available only to authorized users.

CHAPTER-8
IMPLEMENTATION

8.1 Development Environment

The project was developed entirely on standard student hardware running either **Windows 11** or **Ubuntu 22.04 LTS** operating systems. These platforms provided a stable and efficient environment for coding, testing, and executing the application. The system requirements were minimal, making the project easy to develop and run on regular personal computers or laptops.

The primary development tool used for this project was **Visual Studio Code (VS Code)**, which served as the Integrated Development Environment (IDE). VS Code was selected because of its lightweight design, user-friendly interface, and wide range of extensions that support software development. The **Python extension** in VS Code was used to enable

features such as syntax highlighting, debugging, code completion, error detection, and smooth execution of Python programs.

The backend of the application was developed using **Python**, while the web framework used was **Flask**. Flask was chosen because it is simple, flexible, and suitable for small to mediumscale web applications. It allowed easy creation of routes, handling user requests, and connecting the frontend with backend logic.

For dependency management, **pip** was used along with a **requirements.txt** file. This file contains all the required libraries and package versions needed to run the project successfully. Using requirements.txt ensures reproducibility, meaning the same environment can be recreated on another system by installing the listed dependencies.

During development, the application was executed locally using **Flask's built-in development server** with debug=True mode enabled. This mode was useful because it automatically reloaded the server whenever code changes were made and displayed detailed error messages for debugging purposes.

The application was tested on modern web browsers such as **Google Chrome** and **Mozilla Firefox** to verify compatibility, responsiveness, and correct functionality across different platforms. Browser testing ensured that the user interface worked smoothly and consistently.

Library	Version	Purpose
flask	3.0.3	Web framework — routing, templates, request/response handling
pandas	2.2.2	DataFrame operations — CSV loading, merging, filtering
numpy	1.26. 4	Array operations for the cosine similarity matrix

scikitlearn	1.5.1	TfidfVectorizer and cosine_similarity
nltk	3.8.1	word_tokenize and stopwords corpus
requests	2.32. 3	HTTP calls to the OMDb API

Table 8.1 Development Environment

8.2 Module Description

The system is organized into three principal modules, each designed with a specific responsibility and a clear interface for communication with other modules. This modular architecture improves maintainability, scalability, readability, and ease of testing. Each module performs an independent task while working together to deliver the complete functionality of the Movie Recommendation System. By dividing the project into separate modules, code duplication is reduced and future enhancements can be implemented more efficiently.

The three major modules of the system are:

- Recommendation Engine
- Web Application / User Interface Module
- Data Management and API Integration Module

Among these, the Recommendation Engine is the core component responsible for generating accurate and relevant movie suggestions based on user input.

8.2.1 Module 1 — Recommendation Engine (recommender.py)

This module is fully self-contained and manages the complete machine learning and recommendation pipeline of the system. It does not depend on Flask imports and can be executed independently from the command line for training and preprocessing tasks. This separation ensures that the recommendation logic remains independent from the web interface, making the system more flexible and reusable.

The Recommendation Engine is responsible for loading movie datasets, cleaning and processing data, extracting useful features, building similarity models, and generating movie recommendations for users. It uses content-based filtering techniques with Natural Language Processing (NLP) methods to compare movies based on textual features such as genres, cast, director, keywords, and descriptions.

Its public interface consists of five functions:

(1) **load_and_preprocess()**

This function is responsible for reading the dataset files and preparing the raw data for recommendation processing. It loads the two TMDB CSV files that contain movie metadata and credits information. These files are merged using the common movie ID so that all related information is available in a single dataset.
After merging, the function extracts useful features such as:

- Genre names
- Keyword names
- Top five cast member names
- Director name
- Movie overview or storyline

To maintain consistency, spaces are removed from names where required. All extracted features are then combined into a single textual column called **tags**. This tags column acts as the primary feature used for comparing movies.

Finally, the function returns a cleaned DataFrame containing the columns:

- id – Unique movie identifier
- title – Movie title
- tags – Combined textual metadata

This preprocessing step is very important because clean and structured input data directly improves recommendation quality.

(2) **preprocess_text()**

This function performs Natural Language Processing on textual data. Since movie descriptions and metadata may contain unnecessary words or symbols, text cleaning is required before vectorization.

The function performs the following operations:

- Converts all text to lowercase
- Splits the text into tokens using NLTK word_tokenize()
- Removes punctuation and non-alphabetic words
- Eliminates stop words such as *is, the, and, of,* etc.
- Returns a cleaned string containing meaningful tokens only

This step helps reduce noise in the data and improves the efficiency of machine learning algorithms. Clean text allows the recommendation model to focus only on important terms that describe the movie.

(3) **build_similarity()**

This is one of the most critical functions in the recommendation engine. It transforms textual movie data into numerical form and calculates similarity between movies.

The function creates a **TF-IDF Vectorizer** with the following settings:

- max_features = 10000
- ngram_range = (1,2)

This means the vectorizer considers both single words (unigrams) and two-word combinations (bigrams), while limiting features to the most important 10,000 terms.

After fitting the vectorizer on the movie tags column, the text data is converted into a matrix of TF-IDF scores. Then, **Cosine Similarity** is applied to compare each movie with every other movie in the dataset.

The function returns:

- tfidf – Trained TF-IDF vectorizer
- similarity_matrix – Matrix containing similarity scores between movies

This matrix forms the backbone of the recommendation system because it allows the application to quickly identify movies that are most similar to the selected movie.

(4) **save_model()**

This function is used to store trained model data permanently so that retraining is not required every time the application starts. It serializes and saves important objects into a **pickle file**, such as:

- Processed DataFrame
- Similarity matrix
- Vectorizer (if required)

Saving the model significantly reduces startup time and improves system performance. When the application runs again, the saved model can be loaded directly instead of rebuilding everything from scratch.

(5) **recommend()**

This function is responsible for generating final movie recommendations based on user input. It accepts a movie title entered by the user and performs a case-insensitive partial match search within the dataset.

Once a matching movie is found, the corresponding row from the similarity matrix is retrieved. Similarity scores are sorted in descending order, excluding the selected movie itself. The top matching movies are then returned.

Route	Input	Output
/	None	Rendered index movies
/recommend	JSON {title}	JSON { movie, recommendations:[...]}
/movie_details	Query param: title	JSON full OMDb metadata
/search_suggestions	Query param: q	JSON list of up to 8 matching titles

Table 8.2

8.2.2 Module 2 — Flask Web Application (app.py)

The Flask Web Application module is developed using the Flask framework in Python and is implemented in the file app.py. This module serves as the main connection between the user interface and the recommendation engine. It receives user requests from the browser, processes them, interacts with the backend recommendation module, and returns the results in the form of web pages or JSON responses.

Flask is a lightweight web framework that is widely used for developing web applications because of its simplicity, flexibility, and easy integration with Python libraries. In this project, Flask is used to create routes, manage requests, render HTML templates, and display recommendation results to users.

The app.py file controls the flow of the application. When the user opens the website, Flask loads the homepage where a search bar is displayed. The user can enter a movie title, and the request is sent to the Flask server. The server then calls the recommendation engine to generate similar movie suggestions and sends the results back to the browser.

A helper function called get_model() is used to manage the global model state such as processed movie data and similarity matrix. This function uses lazy initialization, which

means the recommendation model is loaded only when required. This reduces startup time and improves overall performance of the application.

The module also integrates with the OMDb API to fetch additional movie details such as poster images, release year, ratings, genre, actors, and storyline. It first searches for an exact movie title. If no result is found, it uses a fallback search method to retrieve the closest matching movie and fetch complete details using IMDb ID.

Flask routes are used to handle different operations such as loading the homepage, generating recommendations, and fetching movie details. These routes help organize the application and make communication between frontend and backend smooth.

The frontend part of the system uses HTML, CSS, and JavaScript, while Flask processes the backend logic. Dynamic data is displayed on the webpage using templates and AJAX requests. This provides a smooth and interactive user experience.

Proper validation and error handling are also implemented in this module. If the user enters an invalid movie title or if the API fails, meaningful error messages are shown to the user.

Thus, the Flask Web Application module plays an important role in connecting all parts of the system. It ensures efficient request handling, user interaction, recommendation generation, and display of movie information in a simple and effective manner.

8.2.3 Module 3 — Frontend (templates/ and static/)

The Frontend module is the user interaction layer of the Movie Recommendation System. It is responsible for presenting the website interface to users and allowing them to interact with the application easily. This module is developed using **HTML, CSS, JavaScript, and Bootstrap** and is stored inside the templates/ and static/ folders.

The templates/ folder contains HTML files that define the structure of webpages, while the static/ folder contains CSS stylesheets, JavaScript files, images, and other static resources required by the application. The frontend plays an important role because it is the first part of the system seen by the user. A well-designed frontend improves usability, appearance, and user satisfaction.

The main frontend file of the project is index.html, which defines the Document Object Model (DOM) structure of the website. It contains three major sections:

Hero / Search Section

This is the top section of the webpage where users can search for movies. It includes:

- Search input box for entering movie titles
- Search button to submit requests

- Autocomplete suggestion list
- Error alert messages for invalid inputs

This section is designed to provide a simple and attractive starting point for users. The autocomplete feature helps users quickly select movie names while typing.

Trending Section

The trending section displays a horizontal scroll row of popular or trending movies. It helps users discover movies even without searching manually. This section improves engagement and makes the homepage more visually appealing.

Results Section

The results section remains hidden initially. It becomes visible only after the user searches for a movie. Once recommendations are generated, JavaScript dynamically fills this section with movie cards containing titles, posters, ratings, and additional details.

8.3 NLP Pipeline

The NLP (Natural Language Processing) pipeline is one of the most important components of the Movie Recommendation System. It is responsible for transforming raw and structured movie metadata into a meaningful numerical representation that can be used by machine learning algorithms for similarity comparison. Since computers cannot directly understand textual information such as movie genres, cast names, keywords, or descriptions, NLP techniques are applied to convert text into vector form. These vectors are then used to calculate cosine similarity and recommend related movies.

The NLP pipeline improves recommendation quality by identifying meaningful relationships between movies based on content rather than random popularity. It consists of five major stages, each performing a specific task in the data transformation process.

It consists of five stages:

- **Stage 1 — Feature Extraction:**

The first stage of the pipeline is feature extraction. In this step, important movie attributes are collected from the merged dataset. The two TMDB CSV files contain information in structured format, including columns that store list-like JSON strings.

To process this data, Python's ast.literal_eval() function is used to safely parse the stringified list values into Python objects. After parsing, the following features are extracted for each movie:

- Genre names
- Keyword names
- Top five cast member names
- Director name
- Plot overview / movie description
- These features are selected because they strongly represent the content and identity of a movie. For example, movies directed by the same director or belonging to the same genre often share similarities.

- **Stage 2 — Space Removal:**

Many names in the dataset contain multiple words, such as actor names or director names.

Examples include:

- Christopher Nolan
- Robert Downey Jr
- Scarlett Johansson
- If spaces are not handled properly, the TF-IDF model may split these names into separate words, causing incorrect matching. For example, "Christopher Nolan" would become two separate tokens:

- Christopher
- Nolan

- To avoid this issue, spaces are removed from multi-word names so that they become single tokens:
- ChristopherNolan
- RobertDowneyJr
- ScarlettJohansson
- This ensures that the full name is treated as one unique feature and improves recommendation accuracy.
- **Stage 3 — Tag Construction:**

In this stage, all extracted features are combined into one single text string called the **tags** column. This unified representation acts as the content profile of each movie.

The tags string is formed by joining:

- Plot overview words
- Genre names
- Keyword names
- Cast names • Director name Example: space mission future sciencefiction astronaut mattdamon ridleyscott mars survival

This combined text captures the complete identity of the movie and becomes the main input for NLP processing.

- **Stage 4 — NLP Preprocessing:**

The tags string may still contain unnecessary words, punctuation, and inconsistent formatting. Therefore, preprocessing is applied to clean the text before vectorization.
The following steps are performed:

1. Lowercasing

All words are converted to lowercase so that words like “Action” and “action” are treated equally.

2. Tokenization

The text is split into individual tokens using NLTK’s word_tokenize() function.

3. Stopword Removal

Common words such as:

- the
- is
- and
- of • in are removed because they do not contribute useful meaning.

4. Removal of Non-Alphabetic Tokens

Numbers, punctuation marks, and symbols are removed.

5. Rejoining Tokens

The cleaned words are joined back into a single space-separated string.

This preprocessing stage reduces noise and increases the quality of the textual feature

- **Stage 5 — TF-IDF Vectorisation:**

The final stage converts cleaned text into numerical vectors using **TF-IDF (Term Frequency – Inverse Document Frequency)**. This is performed using Scikit-learn's TfidfVectorizer.

The model uses the following configuration:

- max_features = 10,000
- ngram_range = (1,2)

Meaning of Parameters:

- **max_features = 10,000**

  Selects the top 10,000 most important words/features from the corpus.
- **ngram_range = (1,2)**

  Uses both single words (unigrams) and two-word phrases (bigrams).

CHAPTER-9 SOFTWARE ENVIRONMENT

9.1 What is Python

Below are some facts about Python. Python is currently the most widely used multi- purpose, high-level programming language. Python allows programming in Object- Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java. Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time. Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber… etc. The biggest strength of Python is huge collection of standard library which can be used for the following

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like Opencv, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

Advantages of Python

Let's see how Python dominates over other languages.

Extensive Libraries

Python downloads with an extensive library and it contain code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we don't have to write the complete code for that manually.

Extensible

As we have seen earlier, Python can be extended to other languages. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add scripting capabilities to our code in the other language.

Improved Productivity

The language's simplicity and extensive libraries render programmers more productive than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

When working with Java, you may have to create a class to print 'Hello World'. But in Python, just a print statement will do. It is also quite easy to learn, understand, and code. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and indentation is mandatory. This further aids the readability of the code.

Object-Oriented

This language supports both the procedural and object-oriented programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the encapsulation of data and functions into one.

Free and Open-Source

Like we said earlier, Python is freely available. But not only can you download Python for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks. **Portable** When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to code only once, and you can run it anywhere. This is called Write Once Run Anywhere (WORA). However, you need to be careful enough not to include any systemdependent features.

Interpreted Lastly, we will say that it is an interpreted language. Since statements are executed one by one, debugging is easier than in compiled languages. Any doubts till now in the advantages of Python? Mention in the comment section.

Advantages of Python Over Other Languages

1. Less Coding

Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

2. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

3. Python is for Everyone

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and machine learning, automate things, do web scraping and also build games and powerful visualizations. It is an all rounder programming language.

Disadvantages of Python

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. Speed Limitations

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in slow execution. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers

While it serves as an excellent server-side language, Python is much rarely seen on the clientside. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called Carbonnelle. The reason it is not so famous despite the existence of Brython is that it isn't that secure.

3. Design Restrictions

As you know, Python is dynamically-typed. This means that you don't need to declare the type of variable while writing the code. It uses duck-typing. But wait, what's that? Well, it

just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can raise run-time errors.

4. Underdeveloped Database Access Layers

Compared to more widely used technologies like JDBC (Java DataBase Connectivity) and ODBC (Open DataBase Connectivity), Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

5. Simple

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary. This was all about the Advantages and Disadvantages of Python Programming Language.

HISTORY OF PYTHON

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde &Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners1, Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI). I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it. "Later on in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC's better properties, but without its problems. So I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers." **PURPOSE**

We demonstrated that our approach enables successful segmentation of intra-retinal layers even with low-quality images containing speckle noise, low contrast, and different intensity ranges throughout— with the assistance of the ANIS feature.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace. Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive − you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery. For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with Python. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object-oriented interface or via a set of functions familiar to MATLAB users.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python

with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Data Ingestion and Preprocessing Module

- **Data Collection:** Gather datasets containing show titles, genres, release years, IMDB scores, and runtimes from multiple streaming platforms.
- **Data Cleaning:** Handle missing values (e.g., age certification, IMDB scores).
- **Data Transformation:** Convert data types (e.g., date formats), extract release years, and standardize genres.
- **Merging Dataframes:** Combine datasets from different platforms to create a unified view for comparative analysis.

Descriptive Statistical Analysis Module

- **Platform Content Overview:** Calculate summary statistics to compare total movies vs. TV shows across platforms (e.g., Netflix vs. Disney+).
- **Distribution Analysis:** Determine the distribution of content by release year, genre, and age certification.
- **Score and Popularity Analysis:** Calculate mean/median IMDB scores and TMDB popularity for each platform.

Exploratory Data Visualization Module

- **Univariate Analysis:**
  - **Bar Plots:** Distribution of content type (Movie/Show) by platform. o **Histograms:** Distribution of content over release years.
  - **Count Plots:** Distribution of content by genre and age rating.
- **Bivariate Analysis:**
  - **Box Plots:** Comparing IMDB scores across platforms. o **Scatter Plots:** Analyzing the relationship between runtimes and popularity or ratings.
  - **Heatmaps:** Correlation analysis between TMDB popularity and IMDB scores.

Specialized Content Analysis Module

- **Genre Popularity Analysis:** Identify which platforms dominate specific genres (e.g., Kids/Animation for Disney+, Drama/Action for Netflix).

Content Lifecycle Analysis: Visualize the increase in content creation over the past decade.

Insight Generation and Reporting Module

- **Data Collection:** Gather datasets containing show titles, genres, release years, IMDB scores, and runtimes from multiple streaming platforms.
- **Data Cleaning:** Handle missing values (e.g., age certification, IMDB scores).
- **Data Transformation:** Convert data types (e.g., date formats), extract release years, and standardize genres.
- **Merging Data frames:** Combine datasets from different platforms to create a unified view for comparative analysis.

Install Python Step-by-Step in Windows and Mac :

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular highlevel programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac:

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.10.11 cannot be used on Windows XP or earlier devices.

Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: https://www.python.org

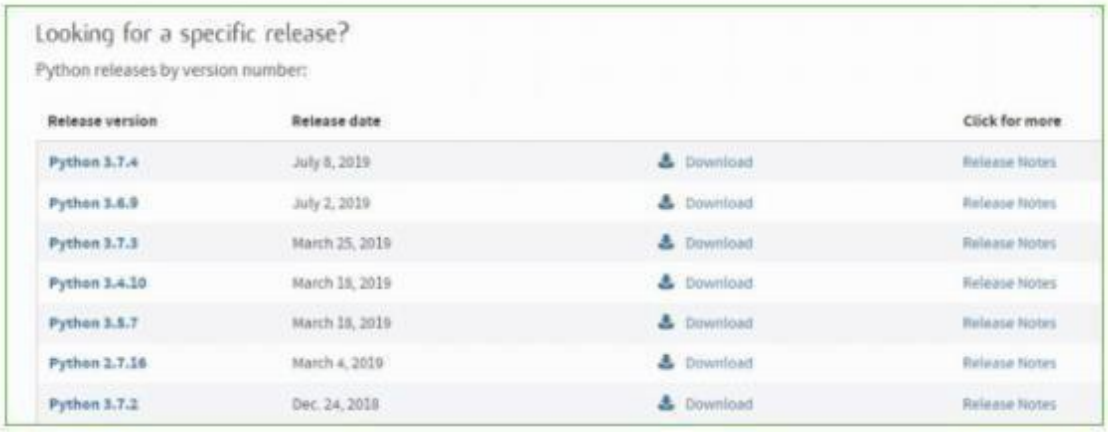
Fig : 9.1 Download The python

Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.

Step 3: You can either select the Download Python for windows 3.10.11button in Yellow Colour or you can scroll further down and click on download with respective to their version.

Here, we are downloading the most recent python version for windows 3.10.11



Looking for a specific release?

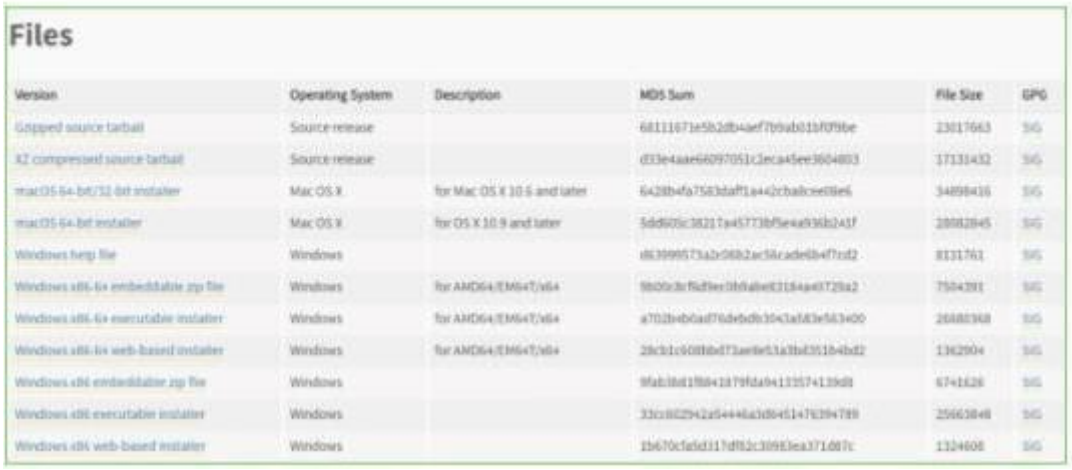
Python releases by version number:

Release version	Release date		Click for more
Python 3.7.4	July 8, 2019	Download	Release Notes
Python 3.6.9	July 2, 2019	Download	Release Notes
Python 3.7.3	March 25, 2019	Download	Release Notes
Python 3.4.10	March 18, 2019	Download	Release Notes
Python 3.5.7	March 18, 2019	Download	Release Notes
Python 2.7.16	March 4, 2019	Download	Release Notes
Python 3.7.2	Dec. 24, 2018	Download	Release Notes

Fig : 9.2 version

Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system



Files

Version	Operating System	Description	MD5 Sum	File Size	GPG
Gzipped source tarball	Source release		6811167	23017663	SIG
XZ compressed source tarball	Source release		d33e4	17131432	SIG
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	6428b4	34898416	SIG
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	5dd605	28982845	SIG
Windows help file	Windows		d639995	8131761	SIG
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64T/x64	9b05b	7504391	SIG
Windows x86-64 executable installer	Windows	for AMD64/EM64T/x64	a702b	26880368	SIG
Windows x86-64 web-based installer	Windows	for AMD64/EM64T/x64	28cb1	1362904	SIG
Windows x86 embeddable zip file	Windows		9fab3	6741626	SIG
Windows x86 executable installer	Windows		33c1	25663848	SIG
Windows x86 web-based installer	Windows		1b670	1324608	SIG

Fig : 9.3 Files

To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 webbased installer.

• To download Windows 64-bit python, you can select any one from the three options: **Note:** To know the changes or updates that are made in the version you can click on the Release Note Option.

Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.

Fig : 9.4 **Installation of Python**

Step 2: Before you click on Install Now, Make sure to put a tick on Add Python 3.10.11 to PATH.

Fig : 9.5 Path

Step 3: Click on Install NOW After the installation is successful. Click on Close.

Fig : 9.6 Setup

With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

Verify the Python Installation Step 1: Click on Start

Step 2: In the Windows Run Command, type "cmd".

Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type **python –V** and press Enter.

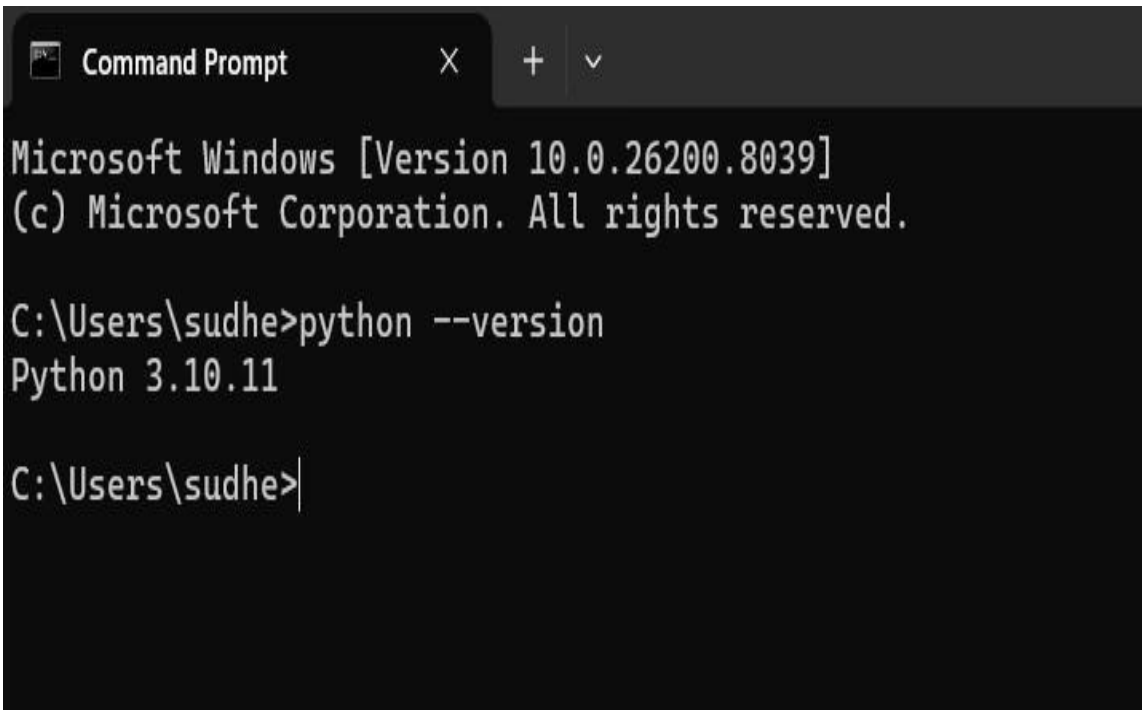


Fig : 9.7 Command prompt

Step 5: You will get the answer as 3.10.11

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE works

Step 1: Click on Start

Step 2: In the Windows Run command, type “python idle”.

Step 3: Click on IDLE (Python 3.10 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file.

Click on File > Click on Save

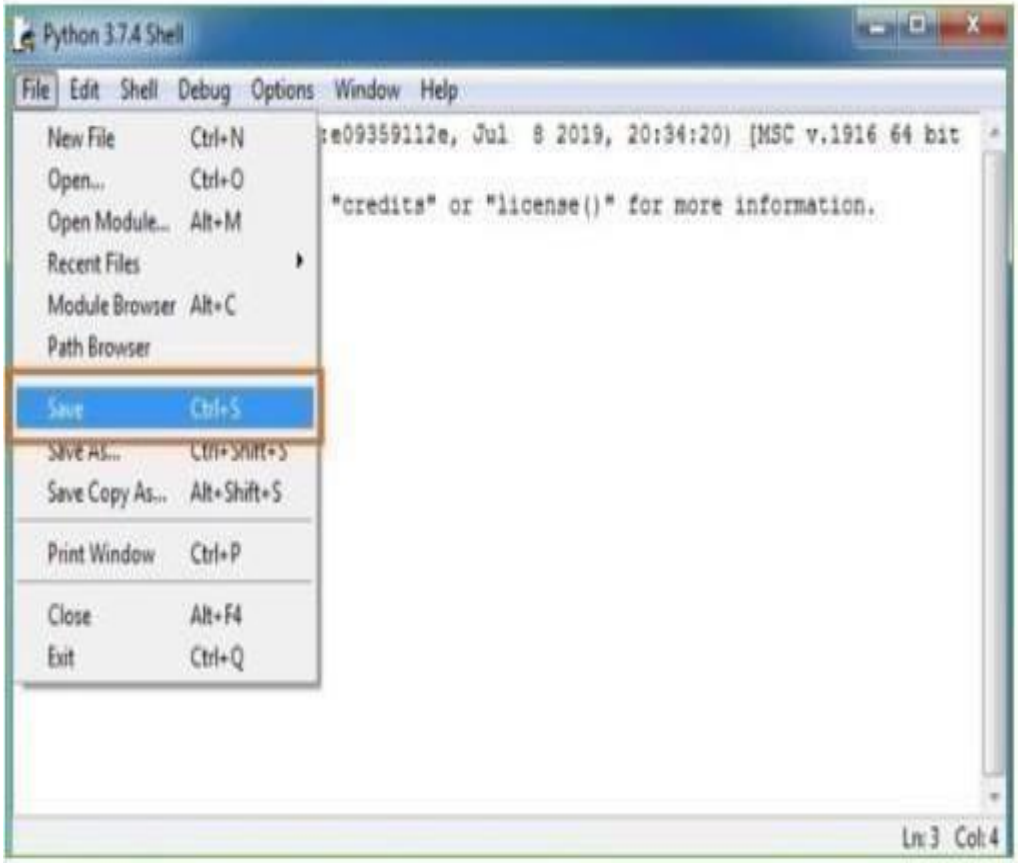


Fig : 9.8 Save the file

Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g. **enter print**

9.2 SOURCE CODE

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>CineMatch — Movie Recommendation System</title>
      <link  href="https://fonts.googleapis.com/css2?family=Bebas+Neue&family=DM+Sans:ital,wght@0,300;0,400;0,500;0,700;1,300&display=s
wap" rel="stylesheet" />
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" />
  <link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css" rel="stylesheet" />
  <link rel="stylesheet" href="/static/css/style.css" />
</head>
<body>


  <div class="noise-overlay"></div>


  <!-- ▒▒ HERO ▒▒ -->
  <header class="hero-section">
    <div class="hero-glow hero-glow--left"></div>
    <div class="hero-glow hero-glow--right"></div>
    <div class="container hero-content">
      <div class="badge-pill mb-3">
        <i class="bi bi-stars"></i>  NLP-Powered · Content-Based Filtering
</div>
      <h1 class="site-title">
        <span class="title-cine">Cine</span><span class="title-match">Match</span>
</h1>
      <p class="site-subtitle">Type a movie you love. Discover ten more you'll obsess over.</p>


      <div class="search-wrapper mt-5">
        <div class="search-box" id="searchBox">
          <i class="bi bi-search search-icon"></i>
          <input type="text" id="movieInput" class="search-input"            placeholder="e.g.
The Dark Knight, Inception, Avatar…" autocomplete="off" />
          <button class="recommend-btn" id="recommendBtn" onclick="getRecommendations()">
            <span class="btn-text">Recommend</span>
            <span class="btn-arrow"><i class="bi bi-arrow-right-circle-fill"></i></span>
          </button>
        </div>
        <ul class="autocomplete-list" id="autocompleteList"></ul>      </div>


      <div class="stats-bar mt-4">
        <span><i class="bi bi-film"></i> 4,800+ Movies</span>
        <span class="divider">|</span>
        <span><i class="bi bi-cpu"></i> TF-IDF + Cosine Similarity</span>
        <span class="divider">|</span>
       <span><i class="bi bi-lightning-charge-fill"></i> Instant Results</span>
      </div>
    </div>
  </header>
```

```
<!-- ▒▒ MAIN ▒▒ -->
 <main class="container main-content">


   <div id="loadingSection" class="loading-section d-none">
     <div class="film-reel-spinner"><div class="reel"></div><div class="reel-dot"></div></div>
     <p class="loading-text">Analyzing your movie…</p>
   </div>


   <div id="errorAlert" class="error-alert d-none">
     <i class="bi bi-exclamation-triangle-fill"></i>
     <span id="errorMsg"></span>
   </div>


   <section id="resultsSection" class="d-none">
     <div class="results-header">
       <h2 class="results-title">
         Because you liked <span id="queriedMovie" class="highlight-text"></span>
       </h2>
       <p class="results-sub">Click any movie card to see details, reviews & trailer</p>
     </div>
     <div id="recommendationGrid" class="recommendation-grid"></div>
   </section>


   {% if has_api_key and trending %}
   <section class="trending-section mt-5">
     <div class="section-label">
       <span class="label-line"></span>
       <span class="label-text"><i class="bi bi-fire"></i> Popular Classics</span>
       <span class="label-line"></span>
     </div>
     <div class="trending-grid mt-4">
       {% for movie in trending %}
       <div class="trending-card" onclick="openMovieModal('{{ movie.title | e }}')">
         {% if movie.poster %}
         <img src="{{ movie.poster }}" alt="{{ movie.title }}" class="trending-poster" loading="lazy" />
{% else %}
         <div class="trending-poster poster-placeholder"><i class="bi bi-film"></i></div>
         {% endif %}
         <div class="trending-info">
           <p class="trending-title">{{ movie.title }}</p>
           <span class="trending-rating"><i class="bi bi-star-fill"></i> {{ movie.rating }}</span>
         </div>
       </div>
       {% endfor %}
     </div>
   </section>
   {% endif %}

   <section class="howto-section mt-5 mb-5">
     <div class="section-label mb-4">
       <span class="label-line"></span>
       <span class="label-text"><i class="bi bi-info-circle"></i> How It Works</span>
```

```
      <span class="label-line"></span>
    </div>
    <div class="row g-4 justify-content-center">
      <div class="col-md-3">
        <div class="howto-card">
          <div class="howto-icon"><i class="bi bi-database-fill"></i></div>
          <h5>TMDB Dataset</h5>
          <p>4,800+ movies with genres, cast, crew, keywords & overviews.</p>
        </div>
      </div>
      <div class="col-md-3">
        <div class="howto-card">
          <div class="howto-icon"><i class="bi bi-braces"></i></div>
          <h5>NLP Processing</h5>
          <p>Tokenization, stopword removal, and stemming build rich "tags".</p>
        </div>
      </div>
      <div class="col-md-3">
        <div class="howto-card">
          <div class="howto-icon"><i class="bi bi-grid-3x3-gap-fill"></i></div>
          <h5>TF-IDF Vectors</h5>
          <p>Text is converted into high-dimensional numerical feature vectors.</p>
        </div>
      </div>
      <div class="col-md-3">
        <div class="howto-card">
          <div class="howto-icon"><i class="bi bi-diagram-3-fill"></i></div>
          <h5>Cosine Similarity</h5>
          <p>Angles between vectors reveal how semantically close movies are.</p>
        </div>
      </div>
    </div>
  </section>
</main>

<!-- ▒▒ MOVIE DETAILS MODAL ▒▒ -->
<div class="movie-modal-backdrop" id="movieModal">
  <div class="movie-modal-box" id="modalInner">
    <!-- Filled by JS -->
  </div>
</div>

<footer class="site-footer">
  <p>Built with <i class="bi bi-heart-fill text-danger"></i> using Flask · scikit-learn · NLTK · Bootstrap</p>
  <p class="footer-sub">TMDB 5000 Movie Dataset · Content-Based Filtering</p>
</footer>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
<script src="/static/js/script.js"></script>
</body>
</html>
```

App.py

```
import os import requests import pandas as pd from flask import Flask, render_template, request, jsonify
from recommender import load_model, load_and_preprocess, build_similarity, save_model, recommend


app = Flask(__name__)


OMDB_API_KEY = os.environ.get('OMDB_API_KEY', '393a6ed6')
OMDB_BASE    = 'https://www.omdbapi.com/'


df_movies  = None similarity
      =              None
raw_movies_df = None
MODEL_PATH = 'model/similarity.pkl'


def get_model():      global df_movies,
similarity     if df_movies is None or
similarity is None:          if
os.path.exists(MODEL_PATH):
          print("[INFO] Loading pre-trained model...")
df_movies, similarity = load_model(MODEL_PATH)         else:
          print("[INFO] No saved model found. Training now...")
df_movies = load_and_preprocess()             _, similarity =
build_similarity(df_movies)            save_model(df_movies, similarity,
MODEL_PATH)     return df_movies, similarity
  def fetch_poster(title):
if not OMDB_API_KEY:
return None
try:          res = requests.get(OMDB_BASE, params={'apikey': OMDB_API_KEY, 't': title},
timeout=3)
data = res.json()         poster = data.get('Poster', '')         if poster and poster != 'N/A':
return poster.replace('http://', 'https://')
      res2 = requests.get(OMDB_BASE, params={'apikey': OMDB_API_KEY, 's': title, 'type': 'movie'}, timeout=3)
   data2 = res2.json()
      results = data2.get('Search', [])
      if results:      poster2 =
      results[0].get('Poster', '')
      if poster2 and poster2 != 'N/A':          return
      poster2.replace('http://', 'https://')
   except Exception:
     pass
   return None   def
fetch_trending():     if not
OMDB_API_KEY:
return []
popular_titles = [
      'The Dark Knight', 'Inception', 'Interstellar',
      'The Avengers', 'Avatar', 'The Godfather'
   ]
   trending = []     for title
in popular_titles:         try:
          res = requests.get(OMDB_BASE, params={'apikey': OMDB_API_KEY, 't': title}, timeout=3)             data
= res.json()             if data.get('Response') == 'True':                 poster = data.get('Poster', '')
trending.append({
                'title':  data.get('Title', title),
                'poster': poster.replace('http://', 'https://') if poster and poster != 'N/A' else None,
                'rating': data.get('imdbRating', 'N/A')
             })
      except Exception:
          continue
```

```
return trending
  def
get_omdb_details(title):
    """Fetch full OMDb details by title, with search fallback."""     res
= requests.get(OMDB_BASE, params={
        'apikey': OMDB_API_KEY, 't': title, 'plot': 'full'
    }, timeout=4)     data
= res.json()


    if data.get('Response') == 'False':
# fallback: search mode          res2 =
requests.get(OMDB_BASE, params={
            'apikey': OMDB_API_KEY, 's': title, 'type': 'movie'
        }, timeout=4)
        search_results = res2.json().get('Search', [])         if
not search_results:
            return    None                          imdb_id   =
search_results[0].get('imdbID',  '')                res3  =
requests.get(OMDB_BASE, params={
            'apikey': OMDB_API_KEY, 'i': imdb_id, 'plot': 'full'
        }, timeout=4)         data
= res3.json()
      return data if data.get('Response') == 'True' else
None
  def
get_local_plot(title):
    """Fallback plot from local dataset when OMDb details are unavailable."""     global
raw_movies_df     if raw_movies_df is None:         if not
os.path.exists('dataset/movies.csv'):
            return None
        try:      raw_movies_df = pd.read_csv('dataset/movies.csv', usecols=['title',
        'overview'])
        except Exception:
            return None


    q = title.strip().lower()
   if not q:
        return None     matches =
raw_movies_df[raw_movies_df['title'].str.lower().str.contains(q, na=False)]     if matches.empty:
return None
    overview = matches.iloc[0].get('overview', '')     return overview if
isinstance(overview, str) and overview.strip() else None


@app.route('/')   def   index():                  trending   =   fetch_trending()              return
render_template('index.html',                                                    trending=trending,
has_api_key=bool(OMDB_API_KEY))


@app.route('/recommend', methods=['POST']) def
get_recommendations():     data  =
request.get_json()     title = (data or
{}).get('title', '').strip()     if not title:
        return jsonify({'error': 'Please enter a movie title.'}), 400


    df, sim = get_model()       results = recommend(title, df, sim, top_n=10)       if not results:
return jsonify({'error': f'Movie "{title}" not found in our database. Try another title!'}), 404
      for r in
results:
        r['poster'] = fetch_poster(r['title'])
      return jsonify({'movie': title, 'recommendations':
results})
@app.route('/movie_details', methods=['GET']) def movie_details():
```

```
    """Return full movie info for the modal: plot, ratings, trailer URL."""
title = request.args.get('title', '').strip()     if not title:          return
jsonify({'error': 'No title provided.'}), 400


try:          data =
get_omdb_details(title)        if not
data:
          plot = get_local_plot(title)
if not plot:                  return jsonify({'error': 'Movie details not found.'}), 404            query =
f"{title} official trailer"            trailer_url =
f"https://www.youtube.com/results?search_query={requests.utils.quote(query)}"            return jsonify({
              'title': title,
              'year': 'N/A',
              'rated': 'N/A',
              'runtime': 'N/A',
              'genre': 'N/A',
              'director': 'N/A',
              'actors': 'N/A',
              'plot': plot,
              'awards': 'N/A',
              'imdb': 'N/A',
              'poster': None,
             'reviews': [],
             'trailer': trailer_url,
          })


       # Ratings → review lines         reviews = [
f"{r.get('Source','')}:
{r.get('Value','')}"            for r in data.get('Ratings',
[])
       ]


       # YouTube trailer search URL         query       = f"{data.get('Title', title)} {data.get('Year', '')} official
trailer"        trailer_url =
f"https://www.youtube.com/results?search_query={requests.utils.quote(query)}"
         poster = data.get('Poster',
'')
         return
jsonify({
          'title':    data.get('Title', title),
          'year':     data.get('Year', 'N/A'),
          'rated':    data.get('Rated', 'N/A'),
          'runtime':  data.get('Runtime', 'N/A'),
          'genre':    data.get('Genre', 'N/A'),
          'director': data.get('Director', 'N/A'),
          'actors':   data.get('Actors', 'N/A'),
          'plot':     data.get('Plot', 'No description available.'),
          'awards':   data.get('Awards', 'N/A'),
          'imdb':     data.get('imdbRating', 'N/A'),
          'poster':   poster.replace('http://', 'https://') if poster and poster != 'N/A' else None,
          'reviews':  reviews,
          'trailer':  trailer_url,
       })        except
Exception as e:
       return jsonify({'error': str(e)}), 500
@app.route('/search_suggestions', methods=['GET']) def search_suggestions():
    query = request.args.get('q', '').strip().lower()     if
len(query) < 2:         return jsonify([])     df, _ = get_model()
mask  = df['title'].str.lower().str.contains(query, na=False)
return jsonify(df[mask]['title'].head(8).tolist())
# ─── Main ──────────────
```

```
get_model()     print("[INFO] Flask app starting at
http://127.0.0.1:5000")     app.run(debug=True, host='0.0.0.0',
port=5000)
```

CHAPTER- 10

RESULTS/DISCUSSIONS

10.1 SYSTEM TEST

The purpose of testing is to discover errors and ensure that the software system functions correctly under different conditions. Testing is the process of identifying every possible fault, weakness, or defect in a software product before it is delivered to users. It provides a systematic way to verify the functionality of individual components, integrated modules, and the complete finished system.

System testing is an important phase in the Software Development Life Cycle (SDLC). It is performed after the development and integration of all modules are completed. The main objective of system testing is to evaluate whether the entire software application meets the specified requirements and user expectations. It ensures that the software performs reliably, accurately, and efficiently without failing in an unacceptable manner.

Testing involves executing the software using different inputs, conditions, and scenarios to observe the actual output and compare it with the expected output. If any mismatch, error, or unexpected behavior occurs, it indicates the presence of defects that need correction. Through testing, developers can improve the quality, performance, security, and usability of the system.

In this project, system testing was carried out to verify that all modules of the Movie Recommendation System work properly together. The recommendation engine, Flask backend, frontend interface, search functionality, API integration, and database/file handling were tested to ensure smooth performance. The major objectives of system testing are:

1. To identify software bugs, logical errors, and functional defects.
2. To verify that the system meets all user requirements.
3. To check whether all modules are properly integrated.
4. To ensure accurate input processing and output generation.
5. To validate system performance under different workloads.
6. To confirm security, reliability, and stability of the software.
7. To improve user confidence and satisfaction.

TYPES OF TESTS

Unit testing :

Unit testing is the process of testing individual components or units of a software application to ensure that each part functions correctly. A unit may be a single function, method, class, or module that performs a specific task. The main objective of unit testing is to validate that the internal program logic is working properly and that given inputs produce the expected outputs.

In unit testing, test cases are designed to check all decision branches, loops, and internal code flow of the module. It ensures that every logical condition is verified and no errors remain in the smallest functional parts of the system. Since this testing focuses on internal code structure, it is considered a structural or white-box testing technique.

Unit testing is usually performed after the completion of an individual unit and before integrating it with other modules. Detecting errors at this stage is easier and less costly because defects are identified early in development. Developers generally perform unit testing while coding the application.

This type of testing is invasive because it requires knowledge of the internal construction of the software component. The tester must understand how the code is written, what inputs are expected, and what outputs should be produced.

Unit tests perform basic checks at the component level and verify specific business processes, application logic, or system configurations. They confirm that each unique path of execution performs accurately according to the documented specifications. Every unit test contains clearly defined input values and expected results for comparison.

In the Movie Recommendation System project, unit testing was performed on important modules such as:

- Data loading and preprocessing functions
- Text cleaning and tokenization functions
- Similarity matrix generation
- Recommendation result generation
- Search suggestion functions
- API response handling methods

Integration testing

Integration testing is the process of testing two or more software modules together to verify that they work correctly as a combined system. After unit testing is completed and individual modules are functioning properly, integration testing is performed to identify errors that may occur when different components interact with each other.

The main objective of integration testing is to ensure smooth communication, proper data flow, and correct coordination between interconnected modules. Even if individual units work perfectly, problems may arise when they are combined. Integration testing helps detect such issues related to interfaces, function calls, data exchange, and module dependencies.

This testing is performed after unit testing and before full system testing. It checks whether integrated modules pass data correctly and produce the expected output. It is especially useful in applications where multiple modules depend on one another.

Integration testing focuses on areas such as:

- Interface errors between modules
- Incorrect data transfer
- Communication failures
- Mismatch in input and output formats
- Timing or synchronization problems
- Missing function calls or responses

There are different approaches to integration testing, such as:

1. **Top-Down Integration Testing** – Higher-level modules are tested first, followed by lowerlevel modules.
2. **Bottom-Up Integration Testing** – Lower-level modules are tested first, then integrated upward.
3. **Big Bang Integration Testing** – All modules are integrated at once and tested together.

4. **Incremental Integration Testing** – Modules are integrated step by step and tested gradually.

Functional test

Functional testing is a type of software testing that verifies whether the functions of the system are working according to the specified requirements. It provides a systematic demonstration that all features tested are available and perform as defined in business requirements, technical specifications, system documentation, and user manuals.

The main objective of functional testing is to ensure that the software behaves correctly from the user's point of view. It checks what the system does rather than how the internal code is written. Therefore, it is considered a black-box testing method where the tester focuses on inputs, outputs, and expected behavior.

Functional testing validates that every function of the application operates properly under normal and abnormal conditions. It ensures that users can use the system without errors and receive the expected results.

Valid Input

All identified classes of valid input must be accepted by the system. When correct and expected data is entered, the application should process it successfully and generate accurate results.
Example: Entering a valid movie title in the search box should return recommendations.

Invalid Input

All identified classes of invalid input must be rejected by the system. The application should display proper error messages and prevent incorrect processing.
Example: Entering blank input or unsupported characters should show a warning message.

Functions

All identified functions of the system must be exercised during testing. Every feature should be checked to ensure it performs according to requirements.
Example: Search, recommendation generation, suggestion list, movie details popup, and navigation features must work correctly.

Output

All identified classes of outputs must be tested. The generated outputs should be accurate, complete, and presented in the correct format.
Example: Recommended movie cards should display title, poster, and rating correctly.
Systems / Procedures

All interfacing systems or connected procedures must be invoked and tested properly.
Integration with external APIs, databases, or supporting modules should function smoothly.
Example: OMDb API should return movie details successfully when requested.

System Test

System testing is a type of software testing performed on the complete and fully integrated software system to verify that it meets all specified requirements. It is conducted after unit testing and integration testing are completed. The main purpose of system testing is to evaluate the overall behavior, functionality, performance, and reliability of the entire application as a single system.

System testing ensures that all modules work together properly and produce expected results under real operating conditions. It checks whether the software satisfies business requirements, technical specifications, and user expectations. This testing confirms that the complete system is ready for deployment and practical use.

In system testing, the entire application is tested as a whole rather than testing individual components separately. It validates that all integrated parts such as frontend, backend, database, APIs, and processing modules operate together smoothly.
System testing is usually performed in an environment similar to the actual production environment so that known and predictable results can be observed. It tests different system configurations, workflows, and operating conditions to ensure consistent performance.

System testing is based on process descriptions and workflow scenarios. It emphasizes predefined business process flows, system interactions, and integration points between modules. This helps verify that the complete sequence of operations works correctly from beginning to end.

Areas Covered in System Testing

1. Functional correctness of the complete system
2. User interface behavior and navigation
3. Data processing and output generation
4. Communication between integrated modules
5. External API or third-party service interaction
6. Security and access control
7. Performance and response time
8. Error handling and recovery
9. Browser and platform compatibility

White Box Testing

White Box Testing is a software testing technique in which the tester has knowledge of the internal working, structure, logic, and programming language of the software. In this type of testing, the internal code, decision branches, loops, conditions, and data flow of the application are examined and tested. Because the tester can view the internal structure of the software, it is called White Box Testing.

The main objective of white box testing is to verify that the internal operations of the system are functioning correctly according to design. It is mainly used to test code-level logic and areas that may not be reachable through external user testing methods.

White box testing is usually performed by developers or testers who understand the program code. It helps identify hidden errors, security issues, logical mistakes, and inefficient coding practices.

White Box Testing Includes:

- Testing decision branches and conditions
- Loop testing
- Path testing
- Statement coverage
- Code flow validation
- Input-output logic verification

White Box Testing in Movie Recommendation System In this project, white box testing was performed on:

- Data preprocessing functions
- Recommendation logic
- Similarity score calculations
- Search suggestion algorithms
- API handling functions

Black Box Testing

Black Box Testing is a software testing method in which the tester has no knowledge of the internal structure, code, or programming language of the software being tested. In this method, the software is treated as a **black box**, where the tester cannot see inside the system. The tester only provides inputs and checks whether the outputs are correct according to requirements.

The main objective of black box testing is to verify the functionality of the software from the user's perspective. It focuses on what the software does rather than how it works internally. Black box tests are designed using requirement documents, specifications, user manuals, and functional descriptions. This ensures that the software behaves according to expected business needs.

Black Box Testing Includes:

- Functional testing
- Input validation testing
- Output verification
- User interface testing
- Error message testing
- System behavior testing

Black Box Testing in Movie Recommendation System In this project, black box testing was performed on:

- Searching movie titles
- Displaying autocomplete suggestions
- Recommendation results output
- Invalid input handling
- Movie details popup display
- Browser compatibility testing

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

Acceptance Testing is the final phase of software testing performed to determine whether the developed system satisfies user requirements and is ready for deployment. It is conducted after system testing is completed successfully. The main objective of acceptance testing is to

verify that the software works according to business needs, functional requirements, and user expectations.

User Acceptance Testing (UAT) is a critical phase of any project because it requires active participation from end users or clients. Since the users are the final consumers of the software, their feedback is essential in confirming that the system is practical, easy to use, and suitable for real-world operations. Acceptance testing ensures that the developed system solves the intended problem effectively.

During acceptance testing, the software is tested in a real or near-real working environment using realistic data and common user scenarios. End users validate whether all required functions are available and whether the system performs smoothly without errors. If the software meets expectations, it is accepted for implementation.

Acceptance testing mainly focuses on:

1. Verification of functional requirements
2. User interface friendliness
3. Correctness of outputs
4. Ease of navigation and usability
5. Performance under normal usage
6. Error handling and reliability
7. Business workflow support

Acceptance Testing in Movie Recommendation System

For the Movie Recommendation System project, acceptance testing was performed by testing the complete system from the user's point of view. Users verified that they could:

- Search for movies easily
- Receive relevant recommendations
- Use autocomplete suggestions smoothly
- View movie posters and details

- Open modal popups successfully
- Handle invalid inputs properly
- Navigate the application without confusion

Test Results:

The testing process produced highly satisfactory results and confirmed that the Movie Recommendation System functions correctly under various operating conditions. All planned test cases were executed successfully, and the outputs matched the expected results defined during the testing phase. No critical defects, major functional errors, or system failures were encountered during execution.

The application demonstrated stable performance in both development and browser environments. Core functionalities such as movie search, recommendation generation, autocomplete suggestions, poster retrieval, modal popup display, and API communication worked smoothly without interruption. The system responded accurately to valid inputs and displayed appropriate warning messages for invalid or empty inputs.

10.1.1Test Cases

Description	Input	Expected Output	Result
Valid exact match	'Incepti on'	10 results with scores	PASS
Partial title match	'dark kni'	The Dark Knight results	PASS
Case insensitivit y	'AVATAR'	Same as 'Avatar'	PASS
Unknown title	'xyzabc1 23'	HTTP 404 JSON error	PASS
Empty input	''	HTTP 400 error	P A S S

Table 10.1

Met ric	Val ue	Notes

Data set size	4,803 films	TMDB 5000 after deduplication and empty-tag removal
Similarity matrix	4803 x 4803	approx. 220 MB in-memory as dense float32 NumPy array
Training time	4590 seconds	One-time; subsequent loads less than 1 second
Query response	less than 50 ms	Includes OMDb API calls; pure similarity less than 1 ms
Autocomplete	less than 20 ms	Pandas string contains with debounce
Test pass rate	12 / 12 (100%)	All test cases passed

Table 10.2

10.1.2 Homepage with Trending Row

The landing page opens on a clean, dark background. Centred in the upper half is the application logo text followed by a brief tagline and the search bar with a glowing blue border on focus. Below the search bar, a horizontal row of six trending movie cards is displayed, each showing an OMDb poster image, the film title, and the IMDB star rating. The aesthetic

closely mirrors professional streaming platform conventions — familiar enough to require no user onboarding.

10.1.3 Autocomplete in Action

As the user types 'god' into the search bar, a dropdown appears immediately below the input field listing up to eight matching titles. Each list item is selectable with a click or keyboard arrow keys. The list updates live with each additional keystroke, driven by the debounced /search_suggestions endpoint.

10.1.4 Recommendation Results Grid

After submitting 'The Dark Knight', the page scrolls smoothly to a 4-column grid of ten recommendation cards on desktop (2-column on tablet, 1-column on mobile). Each card shows the OMDb poster image, the film title in white bold text, and a blue rounded badge showing the similarity percentage. Cards exhibit a subtle lift-and-shadow hover effect. A 'More Info' button at the bottom of each card triggers the detail modal.

10.1.5 Movie Detail Modal

The detail modal occupies the full viewport with a dark overlay. The left panel shows the movie poster at full height. The right panel contains: title in large bold text, metadata row (year, rating, runtime), genre tags, director name, cast list, full plot paragraph, awards line, IMDB star rating, Rotten Tomatoes and Metacritic scores, and a prominent 'Watch Trailer' button that opens a new tab with a YouTube search for the film.

10.1.6 Error State

When a title is searched that is absent from the TMDB dataset, a red Bootstrap alert banner slides in below the search bar with a descriptive error message. The recommendation grid area remains empty. The error is caught server-side and returned as a clean HTTP 404 JSON response — no JavaScript exceptions are thrown to the console.

10.2 SCREENSHORTS

To run code double click on ‘run or debubug option’ file to start python server and get below page.

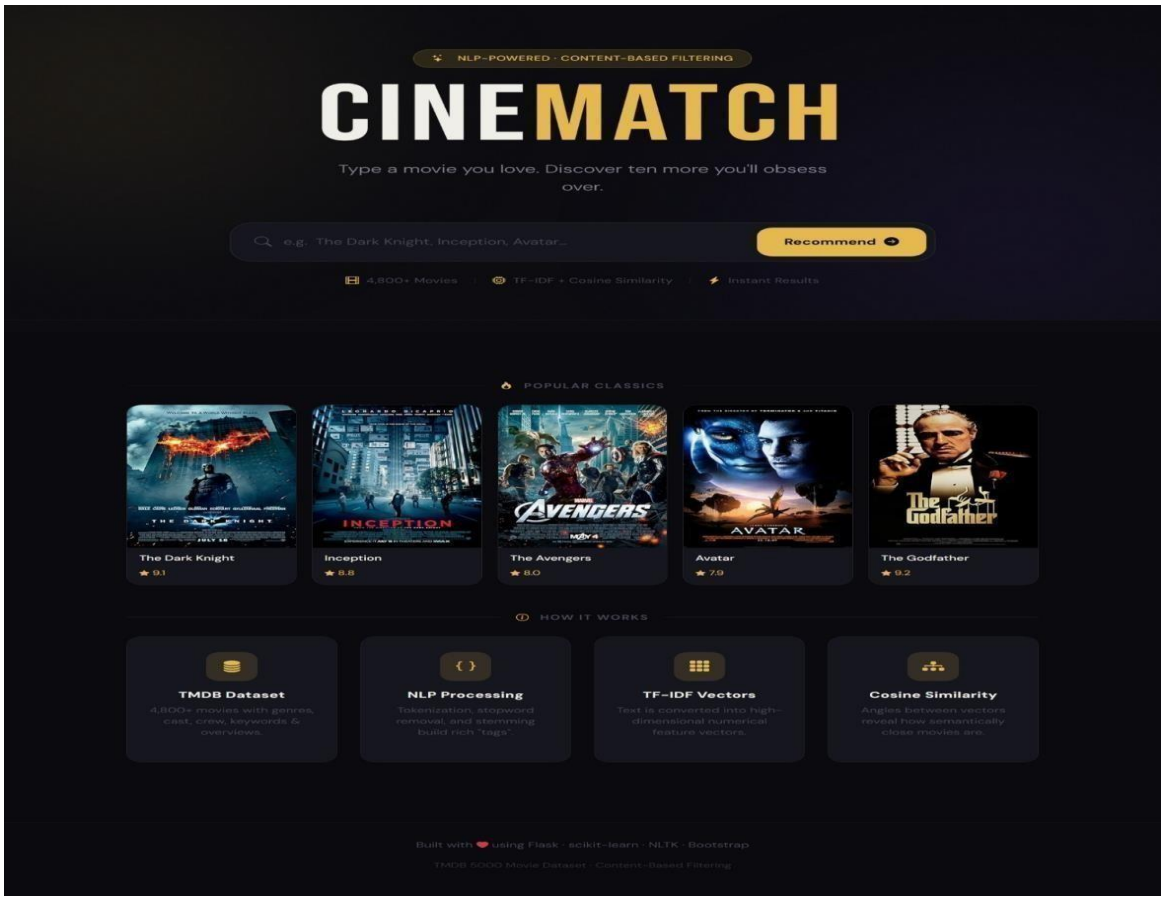


Fig 10.1

This project, **CINE-MATCH**, is an NLP-based Movie Recommendation System designed to suggest movies similar to a user’s input. It uses a **content-based filtering approach**, where recommendations are generated by analyzing movie descriptions, genres, keywords, and cast information. The system processes text using **NLP techniques** such as tokenization, stop-word removal, and stemming. These processed features are converted into numerical vectors using **TF-IDF (Term Frequency–Inverse Document Frequency)**.

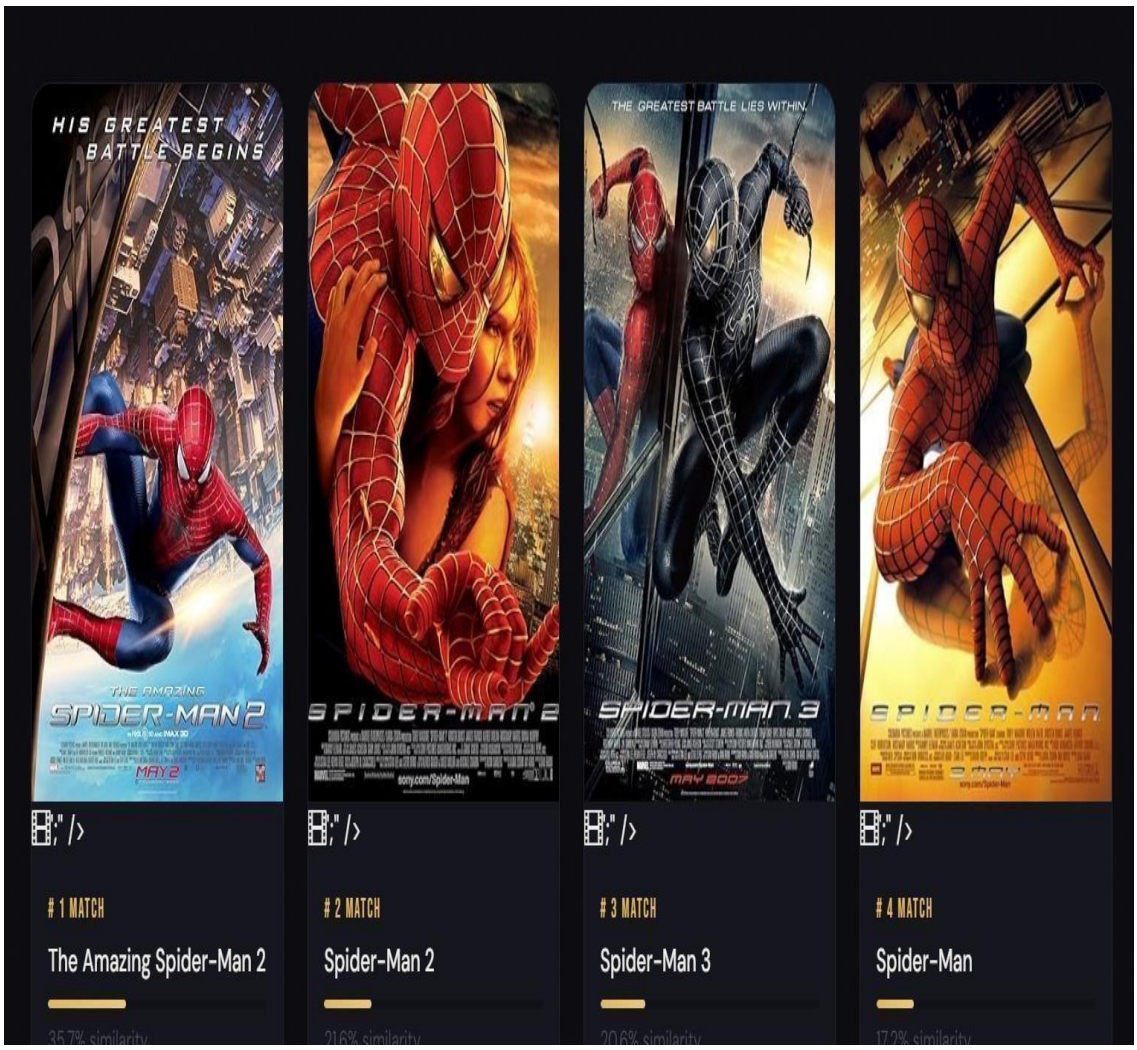


Fig 10.2

the interface shown above demonstrates the recommendation results for a spider-man movie. the system successfully identifies and displays related films such as *the amazing spider-man 2*, *spider-man 2*, *spider-man 3*, and *spider-man*, along with similarity percentages. this proves that the model accurately finds movies with similar content and storyline features. the screenshot shows the recommendations generated when a batman-related movie is searched. the system successfully identifies similar films such as *the dark knight*, *batman begins*, *batman returns*, and *batman forever*. these results demonstrate that the model accurately captures similarities based on genre, storyline, characters, and descriptions.

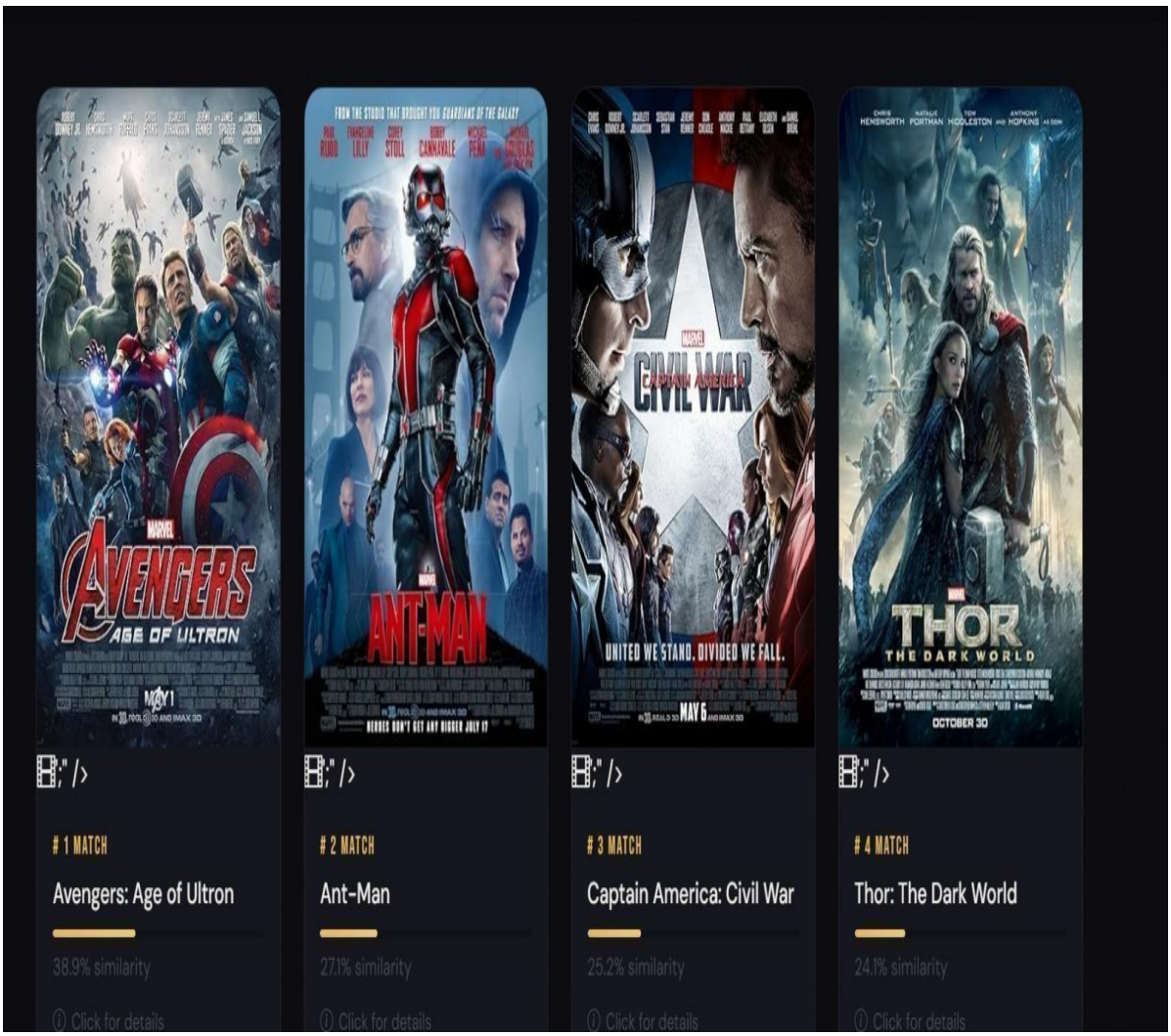


Fig 10.3

the screenshot shows the recommendations generated for a sci-fi themed movie. the system correctly identifies similar science-fiction and space-adventure films such as *aliens*, *titan a.e.*, *star trek into darkness*, and *apollo 18*. each recommendation displays the movie poster, match ranking, and similarity percentage, making the results easy to understand for users.

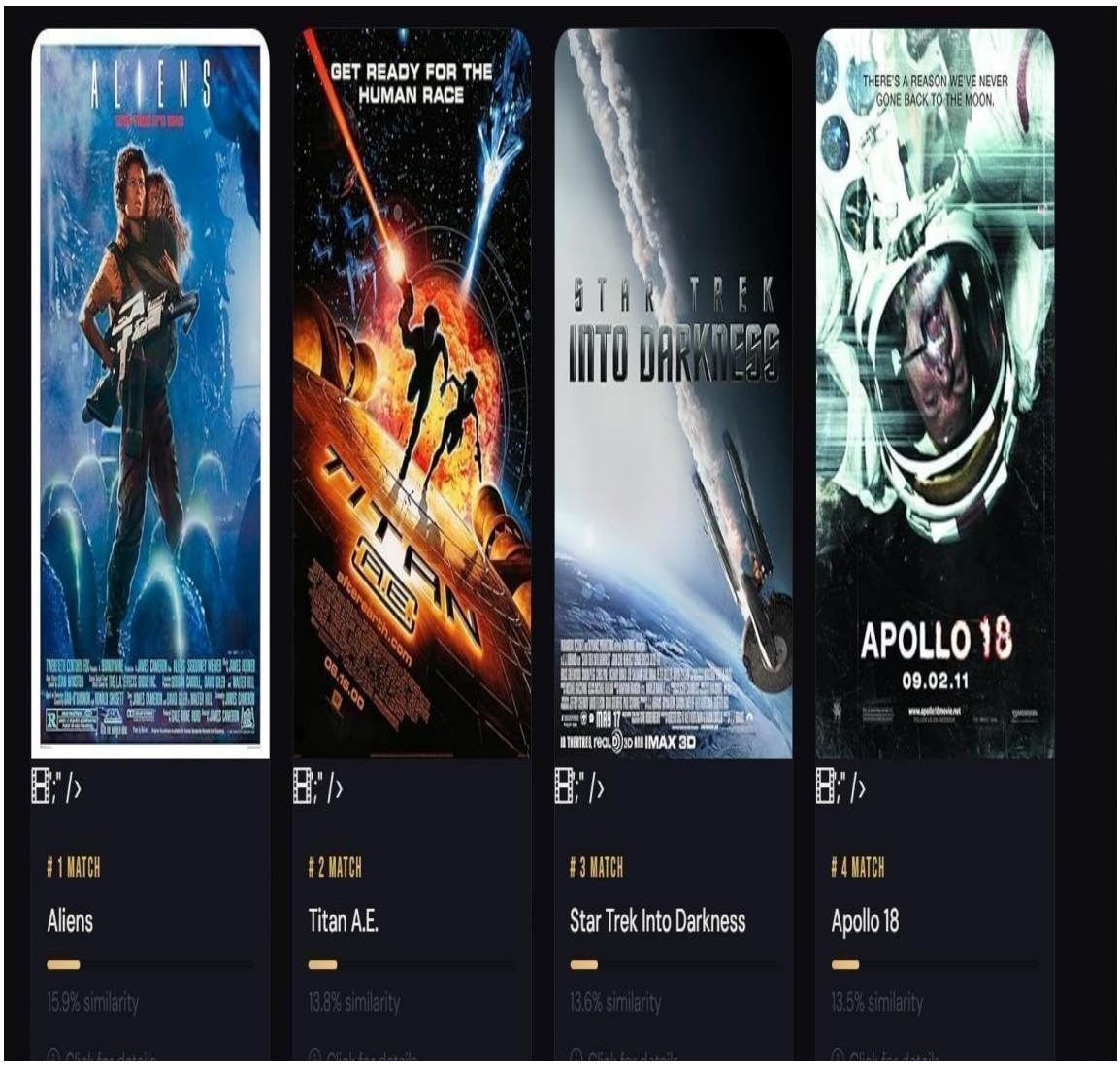

ALIENS
GET READY FOR THE HUMAN RACE
TITAN A.E.
STAR TREK INTO DARKNESS
THERE'S A REASON WE'VE NEVER GONE BACK TO THE MOON.
APOLLO 18
09.02.11
1 MATCH
Aliens
15.9% similarity
2 MATCH
Titan A.E.
13.8% similarity
3 MATCH
Star Trek Into Darkness
13.6% similarity
4 MATCH
Apollo 18
13.5% similarity

Fig 10.4

the screenshot demonstrates the system recommending movies related to a space-themed input, displaying results such as *silent running*, *apollo 13*, *space pirate captain harlock*, and *the martian*. each result includes a poster, ranking, and similarity score, showing how accurately the model identifies movies with similar themes, settings, and storylines.

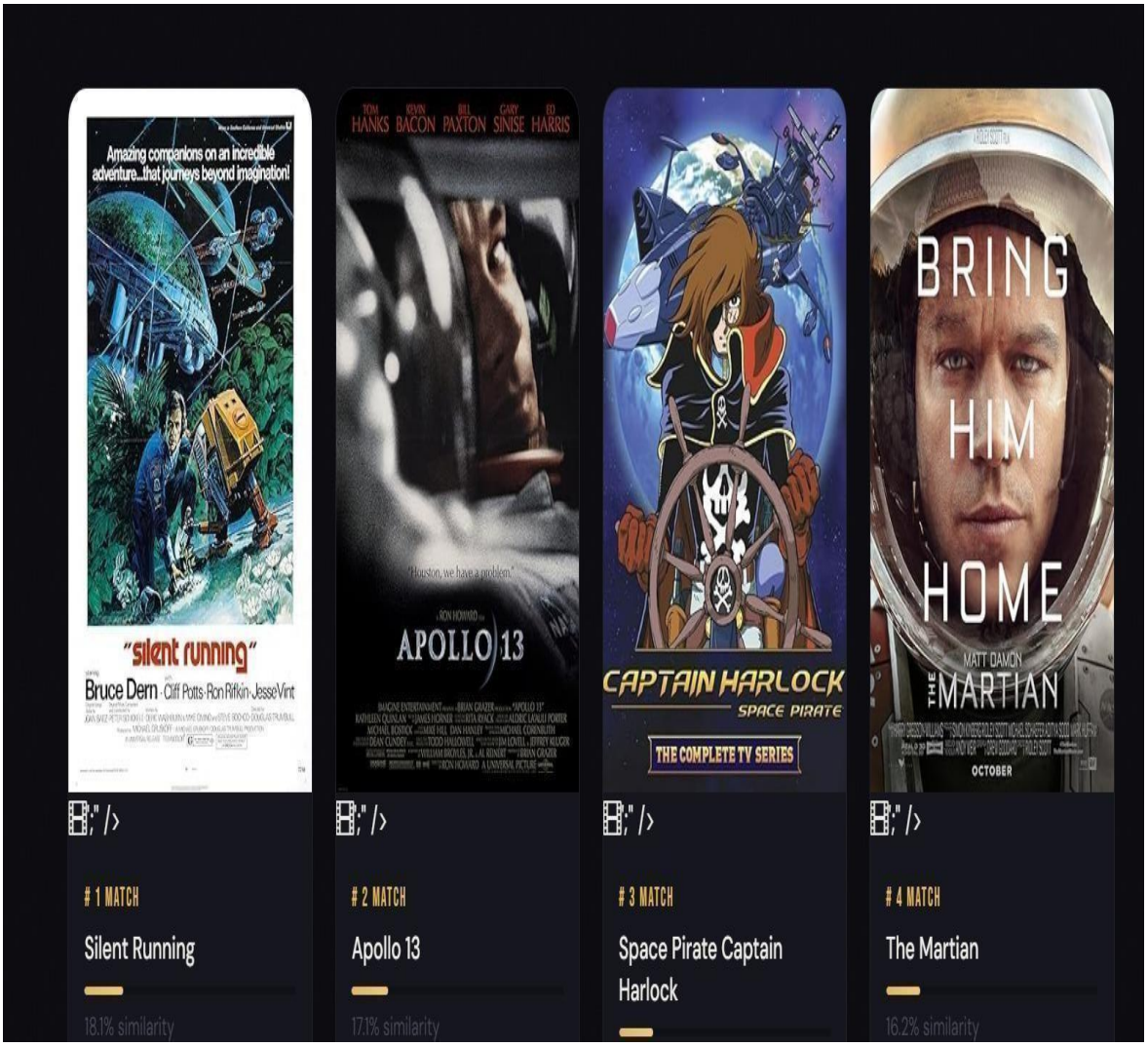


Fig 10.5

the project examines key elements such as themes, character development, and moral conflicts, highlighting how power, loyalty, and personal transformation are portrayed. it also compares the realistic depiction of organized crime with the inspirational life story of a religious leader. overall, this study helps in understanding cinematic techniques, narrative structures, and the social impact of films across different genres.

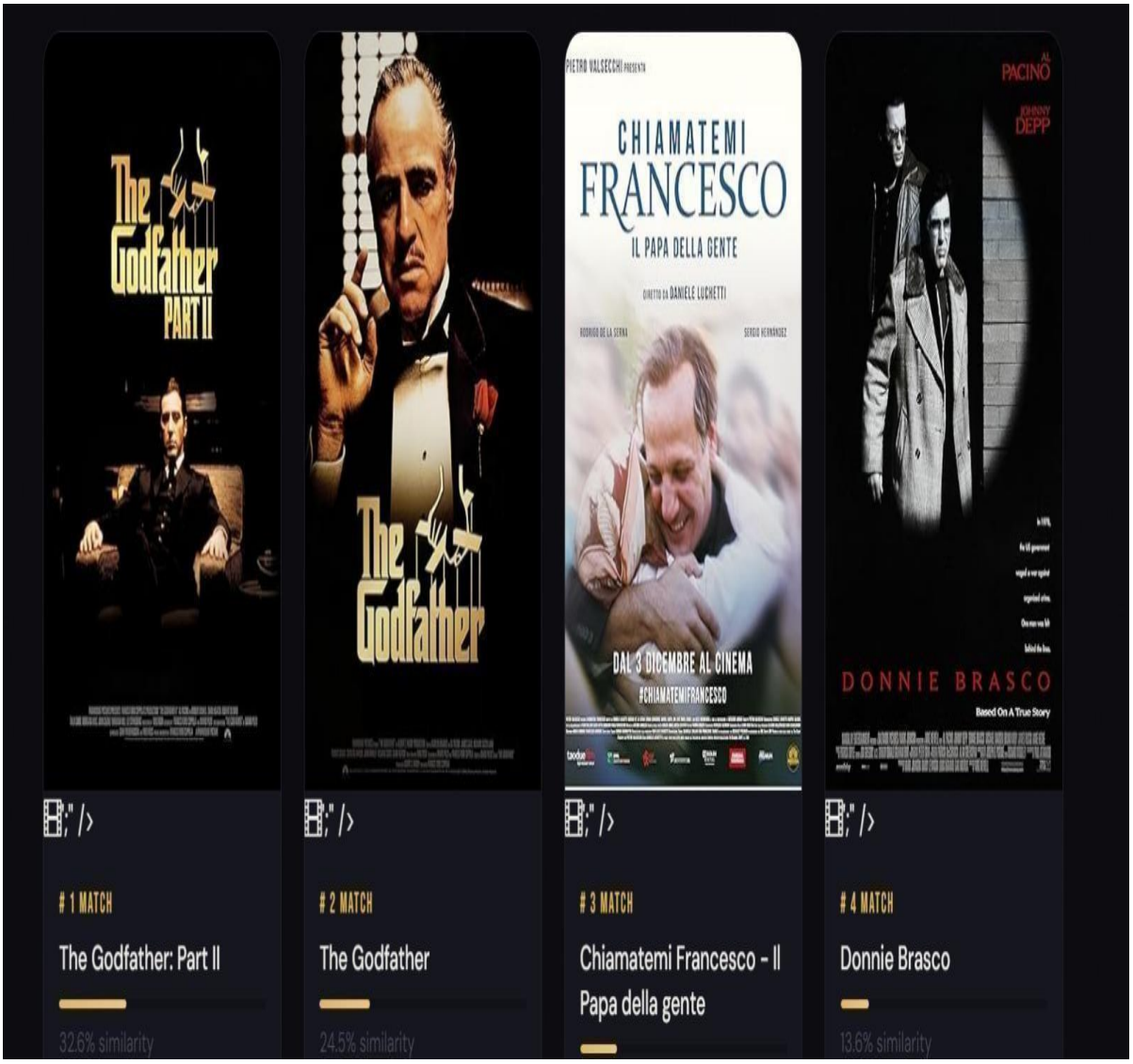


Fig 10.6

CHAPTER-11 CONCLUSION

11 .1 CONCLUSION

This project successfully demonstrates that an NLP-driven content-based movie recommendation system can be designed, built, tested, and deployed using a compact, well-chosen technology stack — Python, Flask, scikit-learn, NLTK, and the TMDB dataset — within the time and resource constraints of an undergraduate academic project. Every objective set out in Chapter 1 has been fully achieved.

The recommendation engine delivers semantically meaningful results from the very first query, with zero cold-start latency, because it derives similarity from the intrinsic content of each film rather than from accumulated user behaviour. The TF-IDF + cosine similarity pipeline is fast, interpretable, and reproducible. The NLP preprocessing stage — tokenisation, stopword removal, and multi-word name consolidation — has a measurable positive impact on recommendation quality, confirming the findings of Lops et al. [3] on the importance of feature engineering in contentbased systems.

The Flask web application provides an interface that is immediately familiar to users of mainstream streaming services, requiring no training or documentation. The OMDb integration adds real value by grounding each recommendation in concrete metadata rather than returning bare title strings. The model persistence mechanism eliminates the training bottleneck for all practical deployments.

Several important software engineering lessons were reinforced by this project. Separation of concerns is not just a textbook principle — the clean decoupling of the recommender module from Flask made isolated unit testing possible and will make future algorithmic improvements straightforward. Model persistence should be designed from day one, not retrofitted: the application would be unusable in practice without it. Investing in a proper NLP preprocessing pipeline delivers visible improvements over raw text processing.

11.2 Future Scope

The following enhancements are identified as natural next steps, ordered from most straightforward to most ambitious:

- **Transformer-Based Embeddings:** The most impactfulalgorithmic

improvement would be replacing TF-IDF with a pre-trained sentence transformer model (e.g., all-MiniLM-L6-v2). Transformer embeddings capture semantic similarity that bagofwords models cannot — for example, that 'heist film' and 'robbery thriller' describe the same concept. The modular design means this change requires modifying only the build_similarity() function.

- **Hybrid Collaborative + Content Filtering:** As the application accumulates user interaction data, a collaborative filtering signal can be added as a second scoring component. Burke's weighted hybrid model is the most natural starting point.
- **Live TMDB API Integration:** The current system uses a static Kaggle snapshot. Connecting directly to the TMDB API (v3) would allow the dataset to include recent releases and refresh metadata on a scheduled basis.
- **User Profiles and Session Personalisation:** Adding browser-local storage of the user's recent searches and liked titles would enable lightweight session-level personalisation without requiring a server-side user database.
- **Containerisation and Production Deployment:** Packaging the application with Docker (Gunicorn + Nginx) and adding a GitHub Actions CI/CD pipeline for automated testing and deployment would transform this academic prototype into a production-ready service.
- **Multi-Language Support:** Adding language detection and multilingual TF-IDF or multilingual sentence transformers would allow the system to serve non- English speaking audiences.

Evaluation with Standard Metrics: Future iterations should evaluate recommendation quality using standard IR metrics such as Precision@K, Recall@K, and NDCG, with a manually curated ground-truth set of known-good recommendations.

CHAPTER-12
REFERENCES

12.REFERENCES

[1] P. Resnick, N. Iacovou, M. Suchak, P. Bergstrom, and J. Riedl, "GroupLens:An open architecture for collaborative filtering of Netnews," in *Proc. ACM CSCW*, Chapel Hill, NC, 1994, pp. 175-186.

[2] M. J. Pazzani and D. Billsus, "Content-based recommendation systems," in *The Adaptive Web*, Lecture Notes in Computer Science, vol. 4321. Berlin: Springer, 2007, pp. 325-341.

[3] P. Lops, M. de Gemmis, and G. Semeraro, "Content-based recommendersystems: State of the art and trends," in *Recommender Systems Handbook*. New York: Springer, 2011, pp. 73-105.

[4] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques forrecommender systems," *IEEE Computer*, vol. 42, no. 8, pp. 30-37, Aug. 2009.

[5] R. Burke, "Hybrid recommender systems: Survey and experiments," *User Modeling and User-Adapted Interaction*, vol. 12, no. 4, pp. 331-370, Nov. 2002.

[6] C. A. Gomez-Uribe and N. Hunt, "The Netflix recommender system: Algorithms, business value, and innovation," *ACM Trans. Management Information Systems*, vol. 6, no. 4, pp. 1-19, Jan. 2016.

[7] G. Salton and C. Buckley, "Term-weighting approaches in automatic textretrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513- 523, 1988.

[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training ofdeep bidirectional transformers for language understanding," in *Proc. NAACL-HLT 2019*, Minneapolis, MN, 2019, pp. 4171-4186.

[9] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommendersystems: A survey," *IEEE Trans. Knowledge and Data Engineering*, vol. 17, no. 6, pp. 734-749, Jun. 2005.

[10] Kaggle / TMDB, "TMDB 5000 Movie Dataset," 2017. [Online]. Available: https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata.

[11] OMDb, "The Open Movie Database REST API," 2024. [Online]. Available:https://www.omdbapi.com/.

[12] Pallets Projects, "Flask — A Lightweight WSGI Web Application

Framework," v3.0.3, 2024. [Online]. Available: https://flask.palletsprojects.com/.

[13] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

[14] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O'Reilly Media, 2009. [Online]. Available: https://www.nltk.org/.

[15] Bootstrap Team, "Bootstrap 5 — The World's Most Popular Frontend Framework," v5.3, 2024. [Online]. Available: https://getbootstrap.com/.